Orthogonal Deep Neural Networks

Shuai Li, Kui Jia, Yuxin Wen, Tongliang Liu, and Dacheng Tao, *Fellow, IEEE*

Abstract—In this paper, we introduce the algorithms of Orthogonal Deep Neural Networks (OrthDNNs) to connect with recent interest of spectrally regularized deep learning methods. OrthDNNs are theoretically motivated by generalization analysis of modern DNNs, with the aim to find solution properties of network weights that guarantee better generalization. To this end, we first prove that DNNs are of local isometry on data distributions of practical interest; by using a new covering of the sample space and introducing the local isometry property of DNNs into generalization analysis, we establish a new generalization error bound that is both scale- and range-sensitive to singular value spectrum of each of networks' weight matrices. We prove that the optimal bound w.r.t. the degree of isometry is attained when each weight matrix has a spectrum of equal singular values, among which orthogonal weight matrix or a non-square one with orthonormal rows or columns is the most straightforward choice, suggesting the algorithms of OrthDNNs. We present both algorithms of strict and approximate OrthDNNs, and for the later ones we propose a simple yet effective algorithm called Singular Value Bounding (SVB), which performs as well as strict OrthDNNs, but at a much lower computational cost. We also propose Bounded Batch Normalization (BBN) to make compatible use of batch normalization with OrthDNNs. We conduct extensive comparative studies by using modern architectures on benchmark image classification. Experiments show the efficacy of OrthDNNs.

Index Terms—Deep neural networks, generalization error, robustness, spectral regularization, image classification

1 Introduction

Deep learning or deep neural networks (DNNs) have been achieving great success on many machine learning tasks, with image classification [46] as one of the prominent examples. Key design that supports success of deep learning can date at least back to Neocognitron [16] and Convolutional neural networks (CNNs) [37], which employ hierarchial, compositional design to facilitate learning target functions that approximately capture statistical properties of natural signals. Modern DNNs are usually over-parameterized and have very high model capacities, yet practically meaningful solutions can be obtained via simple back-propagation training of stochastic gradient descent (SGD) [38], where regularization methods such as early stopping, weight decay, and data augmentation are commonly used to alleviate the issue of overfitting.

Over the years, new technical innovations have been introduced to improve DNNs in terms of architectural design [21], [26], optimization [13], [18], [33], and also regularization [24], [28], which altogether make efficient and effective training of extremely over-parameterized models possible. While many of these innovations are empirically proposed, some of them are justified by subsequent theoretical studies that explain their practical effectiveness. For example, dropout training [24] is explained as an approximate regularization of adaptive weight decay in [3], [54]. Theoretically characterizing global optimality conditions of DNNs are also presented in [30], [60].

The above optimization and regularization methods aim to explain and address the generic difficulties of training DNNs, and to improve efficient use of network parameters; they do not have designs on properties of solutions to which network training should converge. In contrast, there exist other deep learning methods that have favored solution properties of network parameters, and expect such properties to guarantee *good generalization at inference time*. In this work, we specially focus on DNN methods that impose explicit regularization on weight matrices of network layers [50], [55]. For example, Sokolic et al. [50] propose by theoretical analysis a soft regularizer that penalizes Frobenius norm of the Jacobian. More recently, methods that regularize the whole spectrum of singular values and its range for each of networks' weight matrices are also proposed [2], [4], [12], [29], [56], [57]. They achieve clearly improved performance over those without imposing such a regularization. However, many of these methods are empirically motivated, with no theoretical justification on its effect on generalization. We aim to study this theoretical issue in this work.

Motivated by geometric intuitions from isometric mappings [27], we introduce a term of *local isometry* into the framework of generalization analysis via algorithmic robustness [59]. We use an intuitive and also formal definition of *instance-wise variation space* to characterize data distributions of practical interest, and prove that DNNs are of local isometry on such data distributions. More specifically, we prove that for a DNN trained on such a data distribution, a covering based on a linear partition (induced by the DNN) of the input space can be found such that DNN is *locally* linear in each covering ball, where we give bound on the diameters of covering balls in

- *S. Li, K. Jia, and Y. Wen are with the School of Electronic and Information Engineering, South China University of Technology, Guangzhou, Guangdong 510640, China. E-mail: lishuai918@gmail.com, kuijia@scut.edu.cn, wen.yuxin@mail.scut.edu.cn.*
- *T. Liu and D. Tao are with the UBTECH Sydney AI Centre and the School of Computer Science, Faculty of Engineering, University of Sydney, 6 Cleveland St, Darlington, NSW 2008, Australia. E-mail: {tongliang.liu, dacheng.tao}@sydney.edu.au.*

Manuscript received 3 Apr. 2019; revised 4 Oct. 2019; accepted 11 Oct. 2019. Date of publication 21 Oct. 2019; date of current version 4 Mar. 2021. (Corresponding author: Kui Jia.) Recommended for acceptance by M. Varma. Digital Object Identifier no. 10.1109/TPAMI.2019.2948352

0162-8828 © 2019 IEEE. Personal use is permitted, but republication/redistribution requires IEEE permission. See https://www.ieee.org/publications/rights/index.html for more information.

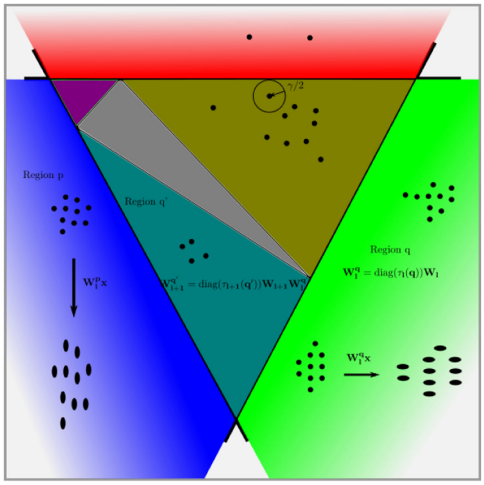


Fig. 1. Illustration for the local δ-isometry of a Deep Neural Network (DNN) with ReLU activation and max pooling functions. Black dots represent training instances in the sample space, and regions coded with different colors represent regions in the instance space that are hierarchically specified by layers of the DNN. The illustration depicts proofs of Lemmas 3.1, 3.2, 3.3, and 3.4. Lemma 3.1 proves that the mapping induced by a linear DNN is of δ-isometry, where δ specified the expansion and contraction properties of the mapping and is determined by singular value spectrums of weight matrices of all the layers. Lemma 3.2 proves that a nonlinear DNN partitions the instance space into increasingly refined regions. As illustrated in the figure, the space is first partitioned into coarser regions (i.e., the center triangle and other three regions color coded as red, blue, and green), an additional layer further partitions some of the coarser regions into sets of smaller regions (e.g., those inside the region of center triangle), and the process goes recursively. Suppose that a region q is created by layer l of the DNN, and in region q, the nonlinear mapping defined by the matrix W_l and activation function reduces to a linear mapping of $W_l^q = \mathrm{diag}(\tau_l(q))W_l$, where $\tau_l(q)$ is a binary vector roughly indicating active neurons, and $\mathrm{diag}(\cdot)$ diagonalizes $\tau_l(q)$. Suppose q' is created by layer $l+1$ at the bottom part of the center triangle, and in region q', the nonlinear mappings of layer l and layer $l+1$ are reduced to a linear mapping of $W_{l+1}^{q'} = \mathrm{diag}(\tau_{l+1}(q'))W_{l+1}W_l^{q'}$, where symbols have similar meanings as described above. The phenomenon enables to find a covering for the sample space, such that in each covering ball that contains training instances, e.g., x, the DNN T defines a transformation that can be characterized as a linear mapping, e.g., $T_{|x}x$. This is proved in Lemma 3.3. Radius of the covering ball is illustrated as $\gamma/2$ in the figure — a radius is acceptable as long as it is less than the smallest distance from any of the training instances to their respective region boundaries. The behaviors of δ-isometry of $T_{|x}$ are also visualized in regions p and q respectively. The transformation of T applied on instances x is different in different regions. As a demonstration, in region p, the transformation vertically elongates the distance between instances, while in region q, it horizontally elongates the distance instead. Lastly, in Lemma 3.4, we prove that by Cauchy interlacing law, a nonlinear DNN is of local δ-isometry within each covering ball specified above.

terms of spectral norms of the DNN's weight matrices. Based on a further proof that for a mapping induced by a linear DNN, degree of isometry is fully controlled by singular value spectrum of each of its weight matrices, we establish our generalization error (GE) bound for (nonlinear) DNNs, and show that it is *both scale- and range-sensitive to singular value spectrum of each of their weight matrices*. An illustration of our proofs is given in Fig. 1. Derivation of our bound is based on a new covering of the sample space, as illustrated in Fig. 2, which enables explicit characterization of GEs caused by both the *distance expansion* and *distance contraction* of locally isometric mappings.

To attain an optimal GE bound w.r.t. the degree of isometry, we prove that the optimum is achieved when each weight matrix of a DNN has a spectrum of equal singular values, among which orthogonal weight matrix or a non-square one with orthonormal rows or columns is the most straightforward choice, suggesting the algorithms of *Orthogonal Deep Neural Networks (OrthDNNs)*. Training to obtain a *strict OrthDNN* amounts to optimizing the weight matrices over their respective Stiefel manifolds, which, however, is very costly for large-sized DNNs. To achieve efficient learning, we propose a simple yet effective algorithm of *approximate OrthDNNs* called *Singular Value Bounding (SVB)*. SVB periodically bounds, in the SGD based training iterations, all singular values of each weight matrix in a narrow band around the value of 1, thus achieving near orthogonality (row- or column-wise orthonormality) of weight matrices. In this work, we also discuss alternative schemes of soft regularization [5], [12], [57] to achieve approximate OrthDNNs, and compare with our proposed SVB. Batch Normalization (BN) [28] is commonly used in modern DNNs, yet it has a potential risk of ill-conditioned layer transform, making it incompatible with OrthDNNs. We propose *Degenerate Batch Normalization (DBN)* and *Bounded Batch Normalization (BBN)* to remove such a potential risk, and to enable its use with strict and approximate OrthDNNs respectively.

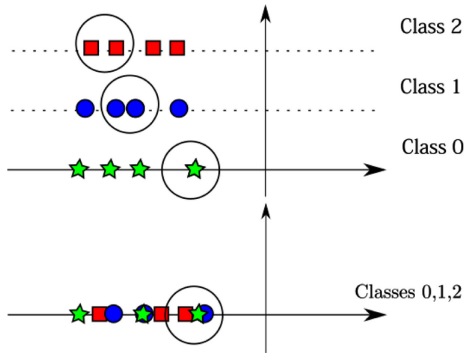


Fig. 2. Illustration of our used covering scheme that includes instances of different labels into same balls. Colored squares, circles, and stars represent instances of different labels, and unfilled (large) circles represent covering balls. *Top:* Existing works [50], [59] usually separate instances of different labels into different covering balls, either by assuming that distances between instances of different labels in the sample space are infinite, or by using covering balls that are small enough not to contain instances of different labels; such a scheme can characterize the generalization errors caused by improper expansion of intra-class variations, but it cannot characterize the errors caused by improper contraction of inter-class differences. *Bottom:* We use a covering scheme that includes instances into covering balls regardless of their labels; it enables characterization of both types of the aforementioned errors, and leads to a possibly tighter generalization error bound in some cases.

To investigate the efficacy of OrthDNNs, we conduct extensive experiments of benchmark image classification [35], [46] on modern architectures [22], [26], [49], [58], [61]. These experiments show that OrthDNNs consistently improve generalization by providing regularization to training of these architectures. Interestingly, approximate OrthDNNs perform as well as strict ones, but at a much lower computational cost. For approximate OrthDNNs, we also compare hard regularization via our proposed SVB and BBN with the alternatives of soft regularization; our results are better than or comparable to those of these alternatives on modern architectures. In some of these studies, we investigate behaviors of our method under learning regimes from small to large sizes of training samples; results confirm the empirical strength of our method, especially for learning problems of smaller sample sizes. We

also investigate robustness of our method against corruptions that are commonly encountered in natural images; our results demonstrate better robustness against such corruptions, and the robustness stands gracefully with increase of corruption severity levels.

1.1 Relations with Existing Works

1.1.1 Generalization Analysis of DNNs

Classical theories of DNNs show that they are universal approximators [6], [25]. However, recent results from Zhang et al. [62] show an apparent puzzle that over-parameterized DNNs are able to shatter randomly labeled training data, suggesting worst-case generalization since test performance can only be at a chance level, while at the same time they perform well on practical learning tasks (e.g., ImageNet classification); the puzzle suggests that traditional analysis of data-independent generalization does not readily apply. They further conjecture [63] that over-parameterized DNNs, when trained via SGD, tend to find local solutions that fall in, with high probability, flat regions in the high-dimensional solution space, which is even obvious when learning tasks are on natural signals; flat-region solutions imply robustness in the parameter space of DNNs, which may further implies robustness in the input data space. Similar argument of flat-region solutions is also presented in [32], although Dinh et al. [14] argue that these flat minima can be equivalently converted as sharp minima without affecting network prediction. Generalization of DNNs is also explained by stochastic optimization. In [20], the notion of uniform stability [10] is extended to characterize the randomness of SGD, and a generalization bound in expectation is established for learning with SGD. The distribution-free stability bound of [20] is improved in [36] via the notion of on-average stability, revealing data-dependent behavior of SGD. To understand practical generalization of DNNs, Kawaguchi et al. [31] argue that independent of the hypothesis set and algorithms used, the learned model itself, possibly selected via a validation set, is the most important factor that accounts for good generalization; a generalization bound w.r.t. validation error is also presented in [31].

To further characterize generalization of DNNs with their weight matrices, Sokolic et al. [50] study DNNs as robust large-margin classifiers via the algorithmic robustness framework [59]. They introduce a notion of average Jacobian, and use spectral norm of the Jacobian matrix to locally bound the distance expansion from the input to the output space of a DNN; spectral norm of the Jacobian is further relaxed as the product of spectral norms of the network's weight matrices, which is used to establish the robustness based generalization bound. Bartlett et al. [7] use a scale-sensitive measure of complexity to establish a generalization bound. They derive a margin-normalized spectral complexity, i.e., the product of spectral norms of weight matrices divided by the margin, via covering number approximation of Rademacher complexity; they further show empirically that such a bound is task-dependent, suggesting that SGD training learns parameters of a DNN whose complexity scales with the difficulty of the learning task.

While both of our bound and that of [50] are developed under the framework of algorithmic robustness [59], our bound is controlled by the whole spectrum of singular values, rather than spectral norm (i.e., the largest singular value) of each of the network's weight matrices, by introducing a term of local isometry into the framework. This also means that in contrast to [7], our bound is both scale- and range-sensitive to singular values of weight matrices. The fact that our bound is scale-sensitive in the sense of [7] implies that for difficult learning tasks, e.g., randomly labeled CIFAR10 [62], spectral norms of weight matrices would go extremely large, causing the diameters of covering balls go extremely small and correspondingly the second term of our bound (cf. Theorem 3.2) that characterizes distribution mismatch between training and test samples dominates, and that the bound becomes vacuous. In contrast, for learning tasks of practical interest, e.g., standard CIFAR10 [35], the spectra of singular values of weight matrices are potentially in a benign range, and the bound is of practical use to inspire design of improved learning algorithms.

1.1.2 Compositional Computations and Isometries of DNNs

Montúfar et al. [42] characterize complexity of functions computable by DNNs and establish a lower bound on the maximal number of linear regions into which a DNN (with ReLU activation) can partition the input space, where the bound is derived by compositional replication of layer-wise space partitioning and grows exponentially with depth of the DNN. Our derivation of the analytic form of region-wise linear mapping (cf. Lemma 3.2) borrows ideas from [42]. Similar compositional derivations for the number of computational paths from the network input to a hidden unit are also presented in [30], [31].

Geometric intuition of isometric mappings has been introduced to improve robustness of deep feature transformation [27], where DNNs are studied as a form of transformation functions. However, their development of robustness bound only uses explicitly the distance expansion constraint of isometric mappings; moreover, their studies are in the context of metric learning and for DNNs, they stay on a general function form, with no indications on how layer-wise weight matrices affect generalization.

1.1.3 Optimization Benefits of Isometry/Orthogonality

Previous works [29], [47], [57] show that orthogonality helps the optimization of DNNs by preventing explosion or vanishing of back-propagated gradients. More specifically, a property of dynamic isometry is studied in [47] to understand learning dynamics of deep linear networks. Pennington et al. [45] extend such studies to DNNs by employing powerful tools from free probability theory; they show that with orthogonal weight initialization, sigmoid activation functions can keep the maximum singular value to be 1 as layers go deeper, and isometry of DNNs can be preserved for a large amount of time during training. However, the analysis on optimization benefits does not explain the gain in test accuracy, i.e., the generalization.

1.1.4 Regularization on Weight Matrices

Wang et al. [55] propose Extended Data Jacobian Matrix (EDJM) as a network analyzing tool, and study how the

spectrum of EDJM affects performance of different networks of varying depths, architectures, and training methods. Based on these observations, they propose a spectral soft regularizer that encourages major singular values of EDJM to be closer to the largest one (practically implemented on weight matrix of each layer). As discussed above, a related notion of average Jacobian is used in [50] to motivate a soft regularizer that penalizes spectral norms of weight matrices.

There exist other recent methods [4], [12], [29], [57] that improve empirical performance of DNNs by regularizing the whole spectrum of singular values for each of networks' weight matrices. This is implemented in [12], [57] as soft regularizers that encourage the product between each weight matrix and its transpose to be close to an identity one. Different from [12], [57], we propose a hard regularization method termed Singular Value Bounding (SVB), which periodically bounds in the training process all singular values of each weight matrix in a narrow band around the value of 1, so that orthonormality of rows or columns of weight matrices can be approximately achieved.

1.2 Contributions

There exists a growing recent interest on using spectral regularization to improve training of DNNs. These methods impose explicit regularization on weight matrices of network layers by penalizing either their spectral norms [50], [55] or the whole spectrums of their singular values [4], [12], [29], [57]. The SVB algorithm proposed in our preliminary work [29] is among the later approach. Most of these methods are empirically motivated with no theoretical guarantees. In the present paper, we focus on theoretical analysis of these methods from the perspective of generalization analysis, and prove a novel GE bound for data distributions of practical interest. We also intensively compare empirical performance of these methods, and present their empirical strengths under various learning scenarios. We summarize our technical contributions as follows.

- We present in this paper a new generalization error bound for DNNs. We first prove that DNNs are of *local isometry* on data distributions of practical interest, where the degree of isometry is fully controlled by singular value spectrum of each of their weight matrices. By using a new covering of the sample space and introducing the local isometry property of DNNs into an algorithmic robustness framework, we establish our GE bound and show that it is *both scale- and range-sensitive to singular value spectrum of each of networks' weight matrices*.
- We prove that the optimal bound w.r.t. the degree of isometry is attained when each weight matrix of a DNN has a spectrum of equal singular values, among which orthogonal weight matrix or a non-square one with orthonormal rows or columns is the most straightforward choice, suggesting the algorithms of *Orthogonal Deep Neural Networks (OrthDNNs)*. In this paper, we also present the algorithmic details of OrthDNNs.
- To address the heavy computation of *strict OrthDNNs*, we propose a novel algorithm called *Singular Value Bounding (SVB)*, which achieves *approximate OrthDNNs* via a simple scheme of hard regularization. We discuss alternative schemes of soft regularization, and compare with our proposed SVB. Batch normalization has a potential risk of ill-conditioned layer transform, making it incompatible with OrthDNNs. We propose *Degenerate Batch Normalization (DBN)* and *Bounded Batch Normalization (BBN)* to remove such a potential risk, and to enable its use with strict and approximate OrthDNNs.

2 Problem Statement

We start by describing the formalism of classification problems that jointly learn a representation and a classifier, e.g., via Deep Neural Networks (DNNs).

2.1 The Classification-Representation-Learning Problem and its Generalization Error

Assume a sample space $\mathcal{Z} = \mathcal{X} \times \mathcal{Y}$, where $\mathcal{X}$ is the instance space and $\mathcal{Y}$ is the label space. We restrict ourselves to classification problems in this paper, and have $\boldsymbol{x} \in \mathcal{X}$ as vectors in $\mathbb{R}^n$ and $y \in \mathcal{Y}$ as a positive integer less than $|\mathcal{Y}| \in \mathbb{N}$. We use $S_m = \{s_i = (\boldsymbol{x}_i, y_i)\}_{i=1}^m$ to denote the training set of size m whose examples are drawn independent and identically distributed (i.i.d.) according to an unknown distribution P. We also denote $S_m^{(x)} = \{\boldsymbol{x}_i\}_{i=1}^m$. Given a loss function $\mathcal{L}$, the goal of learning is to identify a function $f_{S_m} : \mathcal{X} \mapsto \mathcal{Y}$ in a hypothesis space (a class $\mathcal{F}$ of functions) that minimizes the expected risk

$$R(f) = \mathbb{E}_{z\sim P}[\mathcal{L}(f(\boldsymbol{x}), y)],$$

where $z = (\boldsymbol{x}, y) \in \mathcal{Z}$ is sampled i.i.d. according to P. Since P is unknown, the observable quantity serving as a proxy to the expected risk $R(f)$ is the empirical risk

$$R_m(f) = \frac{1}{m}\sum_{i=1}^m \mathcal{L}(f(\boldsymbol{x}_i), y_i).$$

One of the primary goals in statistical learning theory is to characterize the discrepancy between $R(f_{S_m})$ and $R_m(f_{S_m})$, which is termed as *generalization error* — it is sometimes termed as generalization gap in the literature

$$\mathrm{GE}(f_{S_m}) = |R(f_{S_m}) - R_m(f_{S_m})|.$$

In this paper, we are interested in using DNNs to solve classification problems. It amounts to learning a map T, which extracts feature characteristic to a classification task, and minimizing R_m simultaneously. We denote classification with this approach as a *Classification-Representation-Learning* (CRL) problem. We single out the map T because most of the theoretical analysis in this paper resolves around it. Rewriting the two risks by incorporating a map T (we write $\boldsymbol{T}$ when it is instantiated by a DNN), we have

$$R(f, T) = \mathbb{E}_{z\sim P}[\mathcal{L}(f(T\boldsymbol{x}), y)], \tag{1}$$

$$R_m(f, T) = \frac{1}{m}\sum_{i=1}^m \mathcal{L}(f(T\boldsymbol{x}_i), y_i). \tag{2}$$

2.2 Generalization Analysis for Robust Algorithms With Isometric Mapping

The upper bounds of GE are generally established by leveraging on certain measures related to the capacity of

hypothesis space $\mathcal{F}$, such as Rademacher complexity and VC-dimension [41]. These complexity measures capture global properties of $\mathcal{F}$; however, GE bounds based on them ignore the specifically used learning algorithms. To establish a finer bound, one may resort to algorithm-dependent analysis [40], [59]. Our analysis of GE bound in this work is based on the algorithmic robustness framework [59] that has the advantage of conveying information of local geometry. We begin with the definition of robustness used in [59].

Definition 1 ($(K, \epsilon(\cdot))$-robustness). *An algorithm is $(K, \epsilon(\cdot))$-robust for $K \in \mathbb{N}$ and $\epsilon(\cdot) : \mathcal{Z}^m \mapsto \mathbb{R}$, if $\mathcal{Z}$ can be partitioned into K disjoint sets, denoted by $\mathcal{C} = \{C_k\}_{k=1}^K$, such that the following holds for all $s_i = (\boldsymbol{x}_i, y_i) \in S_m, z = (\boldsymbol{x}, y) \in \mathcal{Z}, C_k \in \mathcal{C}$:*

$$\forall s_i = (\boldsymbol{x}_i, y_i) \in C_k, \forall z = (\boldsymbol{x}, y) \in C_k$$
$$\Rightarrow |\mathcal{L}(f(\boldsymbol{x}_i), y_i) - \mathcal{L}(f(\boldsymbol{x}), y)| \leq \epsilon(S_m).$$

The gist of the definition is to constrain the variation of loss values on test examples w.r.t. those of training ones through local property of the algorithmically learned function. Intuitively, if $s \in S_m$ and $z \in \mathcal{Z}$ are "close" (e.g., in the same partition C_k), their loss should also be close, due to the intrinsic constraint imposed by f.

For any algorithm that is robust, [59] proves

Theorem 2.1 ([59]). *If a learning algorithm is $(K, \epsilon(\cdot))$-robust and $\mathcal{L}$ is bounded, a.k.a. $\mathcal{L}(f(\boldsymbol{x}), y) \leq M \ \forall z \in \mathcal{Z}$, for any $\nu > 0$, with probability at least $1 - \nu$ we have*

$$\mathrm{GE}(f_{S_m}) \leq \epsilon(S_m) + M\sqrt{\frac{2K\log(2) + 2\log(1/\nu)}{m}}. \tag{3}$$

To control the first term, a natural approach is to constrain the variation of the loss function. Covering number [34] provides a way to bound the variation of the loss function, and more importantly, it conceptually realizes the actual number K of disjoint partitions.

Definition 2 (Covering number). *Given a metric space $(\mathcal{S}, \rho)$, we say that a subset $\hat{\mathcal{S}}$ of $\mathcal{S}$ is a γ-cover of $\mathcal{S}$, if $\forall s \in \mathcal{S}$, $\exists \hat{s} \in \hat{\mathcal{S}}$ such that $\rho(s, \hat{s}) \leq \gamma$. The γ-covering number of $\mathcal{S}$ is*

$$\mathcal{N}_\gamma(\mathcal{S}, \rho) = \min\{|\hat{\mathcal{S}}| : \hat{\mathcal{S}} \text{ is a } \gamma\text{-covering of } \mathcal{S}\}.$$

In [27], they propose δ-isometry as a desirable property in CRL problem to help control the variation, where δ-isometry is a geometric property of mapping functions.

Definition 3 (δ-isometry). *Given a map T that maps a metric space $(\mathcal{P}, \rho_P)$ to another metric space $(\mathcal{Q}, \rho_Q)$, it is called δ-isometry if the following inequality holds*

$$\forall \boldsymbol{x}, \boldsymbol{x}' \in \mathcal{P}, |\rho_Q(T\boldsymbol{x}, T\boldsymbol{x}') - \rho_P(\boldsymbol{x}, \boldsymbol{x}')| \leq \delta.$$

When T in Eqs. (1) and (2) is of δ-isometry, by using $\rho_Q(T\boldsymbol{x}, T\boldsymbol{x}') \leq \rho_P(\boldsymbol{x}, \boldsymbol{x}') + \delta$ a realization of algorithmic robustness (or GE bound in the form of Theorem 2.1) similar to [27] can be established for DNNs as follows.

Theorem 2.2. *Given an algorithm in a CRL problem, if the Lipschtiz constant of $\mathcal{L} \circ f$ w.r.t. $\boldsymbol{Tx}$ is bounded by A, T is of δ-isometry, and $\mathcal{X}$ is compact with a covering number $\mathcal{N}_{\gamma/2}(\mathcal{X}, \rho)$, then it is $(|\mathcal{Y}|\mathcal{N}_{\gamma/2}(\mathcal{X}, \rho), A(\gamma + \delta))$-robust.*

Remark The result in [27] is $(|\mathcal{Y}|\mathcal{N}_{\gamma/2}(\mathcal{X}, \rho), 2A(\gamma + \delta))$-robust; the factor of 2 in the second term is dropped here due to the fact that in CRL problems, we are not doing metric learning as in [27], which involves two pairs of examples, and we only compare one pair of examples. Its proof under the context of DNN, i.e., the proof of Theorem 3.1, is given in the supplementary material, which can be found on the Computer Society Digital Library at http://doi.ieeecomputersociety.org/10.1109/TPAMI.2019.2948352.

Remark Denote $\rho_Q(T\boldsymbol{x}, T\boldsymbol{x}') \leq \rho_P(\boldsymbol{x}, \boldsymbol{x}') + \delta$ as the *expansion property* of the δ-isometry, and $\rho_Q(T\boldsymbol{x}, T\boldsymbol{x}') \geq \rho_P(\boldsymbol{x}, \boldsymbol{x}') - \delta$ as its *contraction property*. We note that the above theorem is established by only exploiting the expansion property. After proving that DNNs achieve locally isometric mappings in section 3.1, we will show that a better generalization can be derived by considering both the properties.

2.3 Notations of Deep Neural Networks

We study the map $\boldsymbol{T}$ as a neural network. We present the definition of Multi-Layer Perceptron (MLP) here, which captures all ingredients for theoretical analysis and enables us to convey the analysis without unnecessary complications, though we note that the analysis extends to Convolutional Neural Networks (CNNs) almost equally.

A MLP is a map that takes an input $\boldsymbol{x} \in \mathbb{R}^n$ from the space $\mathcal{X}$, and builds its output by recursively applying a linear map $\boldsymbol{W}_l$ followed by a pointwise non-linearity g

$$\boldsymbol{x}_l = g(\boldsymbol{W}_l \boldsymbol{x}_{l-1}), \tag{4}$$

where $l \in \{1, \ldots, L\}$ indexes the layer, $\boldsymbol{x}_l \in \mathbb{R}^{n_l}$, $\boldsymbol{x}_0 = \boldsymbol{x}$, $\boldsymbol{W}_l \in \mathbb{R}^{n_l \times n_{l-1}}$, and g denotes the activation function, which throughout the paper is the Rectifier Linear Unit (ReLU) [19]. Optionally g may include max pooling operator [8] after applying ReLU. We also denote the intermediate feature space $g(\boldsymbol{W}_l \boldsymbol{x}_{l-1})$ as $\mathcal{X}_l$ and $\mathcal{X}_0 = \mathcal{X}$. Each $\mathcal{X}_l$ is a metric space, and throughout the paper the metric is taken as the ℓ_2 norm $||\cdot||_2$, shortened as $||\cdot||$. We compactly write the map of a MLP as

$$\boldsymbol{Tx} = \boldsymbol{W}_L g(\boldsymbol{W}_{L-1} \ldots g(\boldsymbol{W}_1 \boldsymbol{x})).$$

We denote the spectrum of singular values of a matrix $\boldsymbol{W}$ by $\sigma(\boldsymbol{W})$, and $\sigma_{\max}$ and $\sigma_{\min}$ are the maximum and minimum (nonzero) singular values of $\boldsymbol{W}$ respectively. We denote the rank of a matrix $\boldsymbol{W}$ by $r(\boldsymbol{W})$, and the null space of $\boldsymbol{W}$ by $\mathcal{N}(\boldsymbol{W})$. We write the complement of $\mathcal{N}(\boldsymbol{W})$ as $\mathcal{X} - \mathcal{N}(\boldsymbol{W})$.

3 Generalization Bounds of Deep Neural Networks

In this section, we develop GE bounds for CRL problems instantiated by DNNs. We identify two quantities that help control a bound, i.e., δ-isometry of $\boldsymbol{T}$ and the diameter γ of covering balls of $\mathcal{X}$. We show that both of the two quantities can be controlled by constraining the spectrum of singular values of the weight matrix associated with each network layer, i.e., spectrums of singular values of $\{\boldsymbol{W}_i\}_{i=1,\ldots,L}$.

To proceed, we consider in this paper variations of instances in $\mathcal{X}$ that are of practical interest — more specifically, those

that output nonzero vectors after passing through a DNN. We first prove in Lemma 3.1 that in such a variation subspace, mapping induced by a *linear* neural network is of δ-isometry, where δ is specified by the *maximum and minimum* singular values of weight matrices of all the network layers. For a *nonlinear* neural network, where we assume ReLU activation and optionally with max pooling, we consider the fact that it divides the input space $\mathcal{X}$ into a set of regions and within each region, it induces a linear mapping. In Lemma 3.2, we specify the explicit form of region-wise mapping $\boldsymbol{T}_q$, associated with any linear region q, with submatrices of $\{\boldsymbol{W}_i^q\}_{i=1,\ldots,L}$, based on which we prove in Lemma 3.3 that a covering set for $\mathcal{X}$ can be found with a diameter γ of covering balls that is upper bounded by a quantity inversely proportional to the product of maximum singular values of weight matrices of some network layers. With the δ and γ specified in Lemmas 3.1 and 3.3, we further prove in Lemma 3.4 that in the instance-wise variation subspaces considered in this paper, a nonlinear neural network $\boldsymbol{T}$ is of local δ-isometry within each covering ball. The proofs are illustrated in Fig. 1.

To develop a GE bound, we propose a covering scheme that includes instances of different labels into the same balls, thus reducing the size of covering set when compared with that in Theorem 2.1. The covering scheme is illustrated in Fig. 2. We correspondingly characterize both the errors caused by *distance contraction* between instances of different labels and those by *distance expansion* between instances of the same labels. Based on such characterization, we come with our *main result of Theorem 3.2*.

Given the bound in Theorem 3.2, we prove in Lemma 3.5 that the optimal bound w.r.t. δ is obtained when *all singular values of the weight matrix of each network layer are of equal ones*, which inspires a straightforward choice of enforcing all singular values to have the value of 1, and thus the algorithms of OrthDNNs.

3.1 δ-Isometry in Deep Neural Networks

We begin with a few definitions necessary for the subsequent analysis.

Definition 4 (Variation subspace of an instance). *Given an instance $\boldsymbol{x} \in \mathcal{X}$, suppose we are interested in the variation of a set $\mathcal{X}' \subseteq \mathcal{X}$ w.r.t. $\boldsymbol{x}$. We call the linear vector space*

$$\mathrm{span}(\{\boldsymbol{x}' - \boldsymbol{x} | \boldsymbol{x}' \in \mathcal{X}'\}),$$

the variation subspace w.r.t. the instance $\boldsymbol{x}$ of $\mathcal{X}$, shorten as variation subspace of instance $\boldsymbol{x}$.

The definition is to formalize variations of interest of particular instances, thus enabling us to discuss what variations a DNN is able to constrain. Correspondingly, we have the following definition of isometry.

Definition 5 (δ-isometry w.r.t. variation subspace of an instance). *Given a map $\boldsymbol{T}$ that maps a metric space $(\mathcal{P}, \rho_P)$ to another metric space $(\mathcal{Q}, \rho_Q)$, it is called δ-isometry w.r.t. the variation subspace $\mathcal{P}_x$ of instance $\boldsymbol{x}$, if the following holds*

$$\forall \boldsymbol{x}' \in \{\boldsymbol{z} \mid \boldsymbol{z} - \boldsymbol{x} \in \mathcal{P}_x\}, |\rho_Q(\boldsymbol{T}\boldsymbol{x}, \boldsymbol{T}\boldsymbol{x}') - \rho_P(\boldsymbol{x}, \boldsymbol{x}')| \leq \delta.$$

We provide an example here to describe δ-isometry w.r.t. the variation subspace of a linear DNN.

Example 1. In a linear DNN $\boldsymbol{T}$, given an instance $\boldsymbol{x}$, it is δ-isometry w.r.t. variation subspace $\mathcal{X} - \mathcal{N}(\boldsymbol{T})$. This is proved in Lemma 3.1. Thus, for any variation $\delta\boldsymbol{x} \in \mathcal{X} - \mathcal{N}(\boldsymbol{T})$, we have $|\rho(\boldsymbol{T}\boldsymbol{x}, \boldsymbol{T}(\boldsymbol{x} + \delta\boldsymbol{x})) - \rho(\boldsymbol{x}, \boldsymbol{x} + \delta\boldsymbol{x})| \leq \delta$. In this case, the variation subspace $\mathcal{X} - \mathcal{N}(\boldsymbol{T})$ is the same for any instance $\boldsymbol{x}$.

The following lemma specifies for a *linear* DNN the δ-isometry w.r.t. data variations of practical interest.

Lemma 3.1. *Given a linear neural network $\boldsymbol{T}$ and an instance $\boldsymbol{x} \in \mathcal{X}$, if $||\boldsymbol{x}|| \leq b$, i.e., instances are norm bounded, then $\boldsymbol{T}$ is of $2b\max(|\prod_{i=1}^L \sigma_{\max}^i - 1|, |\prod_{i=1}^L \sigma_{\min}^i - 1|)$-isometry w.r.t. the variation subspace $\mathcal{X} - \mathcal{N}(\boldsymbol{T})$ of the instance $\boldsymbol{x}$.* [1]*. We also have $\forall \boldsymbol{x}' \in \{\boldsymbol{z} \mid \boldsymbol{z} - \boldsymbol{x} \in \mathcal{P}_x\}$, $\rho_Q(\boldsymbol{T}\boldsymbol{x}, \boldsymbol{T}\boldsymbol{x}') \leq \rho_P(\boldsymbol{x}, \boldsymbol{x}') + 2b|\prod_{i=1}^L \sigma_{\max}^i - 1|$ and $\rho_Q(\boldsymbol{T}\boldsymbol{x}, \boldsymbol{T}\boldsymbol{x}') \geq \rho_P(\boldsymbol{x}, \boldsymbol{x}') + \rho_P(\boldsymbol{x}, \boldsymbol{x}')(\prod_{i=1}^L \sigma_{\min}^i - 1)$.*

See the proof in the supplementary material, available online.

The above lemma shows that as long as $\boldsymbol{x}$ varies within the complement of the null space of a linear DNN $\boldsymbol{T}$, we can constrain variations induced by the mapping by the specified δ. Outside the space, it is unlikely of practical interest since $\boldsymbol{T}$ discards all the information about the variations.

To further proceed for nonlinear DNNs, we introduce some terminologies from hyperplane arrangement [43], and give definitions to describe the objects of interest exactly.

Definition 6 ((Finite) Hyperplane arrangement). *A finite hyperplane arrangement $\mathcal{A}$ is a finite set of affine hyperplanes in some vector space $\mathcal{X} \equiv \mathbb{K}^n$, where $\mathbb{K}$ is a field and is taken as $\mathbb{R}$ in this paper.*

Definition 7 (Region). *Denote $H \in \mathcal{A}$ an element of the arrangement, a region of the arrangement is a connected component of the complement $\mathbb{R}^n - \bigcup_{H \in \mathcal{A}} H$. The set of all regions is denoted as $\mathcal{R}(\mathcal{A})$, shortened as $\mathcal{R}$ when no confusion exists.*

We set up a labeling scheme for $r \in \mathcal{R}$. Choosing a linear order in $\mathcal{A}$, we write $\mathcal{A} = \{H_i\}_{i=1,\ldots,n}$ and $H_i = \ker \alpha_i$, where $\ker$ denotes the kernel $\{\boldsymbol{x} \in \mathcal{X} | \alpha_i(\boldsymbol{x}) = \langle \boldsymbol{\alpha}_i, \boldsymbol{x} \rangle = 0\}$ and $\boldsymbol{\alpha}_i$ is the normal of hyperplane H_i. Let $\mathbb{J} = \{1, 0\}$, and $\pi_i : \mathbb{J}^n \to \mathbb{J}$ be the projection onto the ith coordinate. Define a map $\tau : \mathcal{X} \to \mathbb{J}^n$ by

$$\pi_i \tau(\boldsymbol{x}) = \begin{cases} 1 & \text{if } \alpha_i(\boldsymbol{x}) > 0 \\ 0 & \text{if } \alpha_i(\boldsymbol{x}) \leq 0. \end{cases}$$

With the scheme, for any $\mathcal{A}$, we would have an index set $\mathcal{T}(\mathcal{A}, \tau)$, shortened as $\mathcal{T}$, such that for any $r \in \mathcal{R}$, it corresponds to a unique element in $\mathbb{J}^n$, denoted as $\tau(r)$. We will use $\tau(\boldsymbol{x})$ —- labeling on elements, and $\tau(r)$ — labeling on regions, interchangely.

Definition 8 (Neuron). *A neuron a_{lk} of a neural network $\boldsymbol{T}$ is a functional defined by*

$$a_{lk}(\boldsymbol{x}) = \pi_k g(\boldsymbol{W}_l g(\boldsymbol{W}_{l-1} \ldots g(\boldsymbol{W}_1 \boldsymbol{x}))),$$

1. Note that the space $\mathcal{X} - \mathcal{N}(\boldsymbol{T})$ does not depend on $\boldsymbol{x}$, and this is a trivial case where the variation subspaces of all instances are the same. We will see nontrivial cases later when dealing with nonlinear neural networks.

where $l \in \{1, \ldots, L\}$ and $k \in \{1, \ldots, n_l\}$. All the neurons at layer l define a map, denoted as

$$a_l(\boldsymbol{x}) = g(\boldsymbol{W}_l g(\boldsymbol{W}_{l-1} \ldots g(\boldsymbol{W}_1 \boldsymbol{x}))).$$

The lemma that follows is mostly an analysis of the domain of a DNN $\boldsymbol{T}$. We begin with the following definition.

Definition 9 (Support of Neuron/DNN). *Given a neuron a_{lk}, the support of a_{lk} is the set of instances in $\mathcal{X}$ that satisfy*

$$\text{supp}(a_{lk}) = \{\boldsymbol{x} \in \mathcal{X} | a_{lk}(\boldsymbol{x}) \neq 0\}.$$

Similarly, the support of a neural network $\boldsymbol{T}$ is the set of instances in $\mathcal{X}$ that satisfy

$$\text{supp}(\boldsymbol{T}) = \{\boldsymbol{x} \in \mathcal{X} | \boldsymbol{T}\boldsymbol{x} \neq 0\}.$$

Lemma 3.2. *A nonlinear neural network $\boldsymbol{T}$ divides $\mathcal{X}$ into a set of regions $\mathcal{Q}$, and within each region $q \in \mathcal{Q}$, $\boldsymbol{T}$ is linear w.r.t. variations of instances as long as they vary within q. We denote the linear mapping at q as $\boldsymbol{T}_q$ and have*

$$\boldsymbol{T}_q = \prod_{i=1}^{L} \boldsymbol{W}_i^q$$
$$\boldsymbol{W}_l^q = \text{diag}(\tau_l(q))\boldsymbol{W}_l,$$

where when layer l does not contain max pooling, and $\tau_l(q)$ is defined as

$$\pi_k \tau_l(q) = \pi_k \tau_l^{\text{relu}}(q) = \begin{cases} 1 & \text{if } a_{lk}(\boldsymbol{x}) > 0, \forall \boldsymbol{x} \in q \\ 0 & \text{if } a_{lk}(\boldsymbol{x}) \leq 0, \forall \boldsymbol{x} \in q. \end{cases}$$

When layer l does contain max pooling, we define $\boldsymbol{W}_l^q$ as

$$\boldsymbol{W}_l^q = \boldsymbol{P}_l \text{ diag}(\tau_l(q))\boldsymbol{W}_l,$$

and $\tau_l(q)$ is defined as

$$\tau_l(q) = \tau_l^{\max}(q)\tau_l^{\text{relu}}(q).$$

In this above definition τ^{relu} is defined as before, and

$$\pi_k \tau_l^{\max}(q) = \begin{cases} 1 & \text{if } k = \text{argmax}_{k \in K} a_{lk}(\boldsymbol{x}), \forall \boldsymbol{x} \in q \\ 0 & \text{otherwise,} \end{cases}$$

where K is the set of indices of neurons being pooled over; $\boldsymbol{P}_l$ is defined as (layer index suppressed)

$$\boldsymbol{P}_{ik} = \begin{cases} 1 & \text{if } k \text{ is the index that is pooled over by } i\text{th pooling area} \\ 0 & \text{otherwise.} \end{cases}$$

$\boldsymbol{P}_l$ could be understood as a matrix that for each pooling area, it sums over the dimension/area being pooled, and since only one dimension of the area is nonzero (due to $\tau_l(q)$), it outputs the maximal value. For clarity and convenience, we would use the definition without max pooling in discussion, and note that all the results present are proved for both definitions.

See the proof in the supplementary material, available online.

Remark. The function τ_l is intuitively a selection function that sets some rows of $\boldsymbol{W}_l$ to zero, and selects a submatrix from it.

Remark. For $q \in \mathcal{Q}$ and $q \notin \text{supp}(\boldsymbol{T})$, by the definition of linearity, $\boldsymbol{T}$ is still linear over q, i.e., the special case of $\boldsymbol{T}\boldsymbol{x} = 0, \forall \boldsymbol{x} \in q$. In this case, $\boldsymbol{x} \in \mathcal{N}(\boldsymbol{T}_q)$.

In the proof for Lemma 3.2, we prove that within each region $q \in Q$, each neuron of a DNN is a linear functional. We summarize the result in the following corollary.

Corollary 3.1. *A nonlinear neural network $\boldsymbol{T}$ divides $\mathcal{X}$ into a set of regions $\mathcal{Q}$, and within each region $q \in \mathcal{Q}$, the kth neuron a_{lk} of the layer l is linear w.r.t. variations of $\boldsymbol{x}$ within q, and we have*

$$a_{lk}^q = \pi_k \prod_{i=1}^{l} \boldsymbol{W}_i^q.$$

With the above lemma, we define the behavior of a DNN at a local area around $\boldsymbol{x} \in \mathcal{X}$ or a local area around a set $B \subset \mathcal{X}$ as below.

Definition 10 (Linear neural network and neuron induced at $\boldsymbol{x} \in \mathcal{X}$ from a nonlinear neural network). *For any given $\boldsymbol{x} \in \mathcal{X}$ with $\boldsymbol{x} \in q \in \mathcal{Q}$, we call the linear neural network $\boldsymbol{T}_q$ the linear neural network induced by a nonlinear neural network $\boldsymbol{T}$ at $\boldsymbol{x}$ — denoting it as $\boldsymbol{T}_{|\boldsymbol{x}}$, the linear neuron a_{lk}^q the linear neuron induced by the nonlinear neuron a_{lk} at $\boldsymbol{x}$ — denoting it as $a_{lk|\boldsymbol{x}}$, and the submatrix of weight matrix of each layer l the submatrix induced by nonlinearity — denoting it as $\boldsymbol{W}_{l|\boldsymbol{x}}$.*

Definition 11 (Linear neural network and neuron induced at subset $B \subset \mathcal{X}$ from a nonlinear neural network). *For any given $B \subset \mathcal{X}$ with $B \subset q \subset \mathcal{Q}$, we call the linear neural network $\boldsymbol{T}_q$ the linear neural network induced by a nonlinear neural network $\boldsymbol{T}$ at B — denoting it as $\boldsymbol{T}_{|B}$, the linear neuron a_{lk}^q the linear neuron induced by the nonlinear neuron a_{lk} at B — denoting it as $a_{lk|B}$, and the submatrix of weight matrix of each layer l the submatrix induced by nonlinearity — denoting it as $\boldsymbol{W}_{l|B}$.*

Lemma 3.3. *For any nonlinear neural network $\boldsymbol{T}$ of L layers, a covering set for $\mathcal{X}$ can be found with a diameter $\gamma = o(S_m, \boldsymbol{T})/(\prod_{i=1}^{l(S_m,\boldsymbol{T})} \sigma_{\max}^i) > 0$, such that for any given $\boldsymbol{x} \in S_m^{(x)}$ and $\{\boldsymbol{x}' \in \mathcal{X} | \|\boldsymbol{x} - \boldsymbol{x}'\| \leq \gamma\}$, $\boldsymbol{T}\boldsymbol{x} - \boldsymbol{T}\boldsymbol{x}' = \boldsymbol{T}_{|\boldsymbol{x}}(\boldsymbol{x} - \boldsymbol{x}')$, where $o(S_m, \boldsymbol{T})$ is a value depending on the training data and network weights, so is $l(S_m, \boldsymbol{T})$ with $1 \leq l(S_m, \boldsymbol{T}) \leq L$ (the dependence is specified in the proof), and $\sigma_{\max}^i$ is the maximum singular value of weight matrix $\boldsymbol{W}_i$ of the ith layer.*

See the proof in the supplementary material, available online.

We come with the local isometry property of DNNs after one more definition.

Definition 12 (γ-cover δ-isometry w.r.t. variation subspace of instance). *Given a map $\boldsymbol{T}$ that maps a metric space $(\mathcal{P}, \rho_P)$ to another metric space $(\mathcal{Q}, \rho_Q)$, and an instance $\boldsymbol{x} \in \mathcal{P}$, it is called γ-cover δ-isometry w.r.t. variation space $\mathcal{P}_{\boldsymbol{x}}$ of $\boldsymbol{x}$, if a γ-cover exists such that the following inequality holds*

$$\forall \boldsymbol{x}' \in \{z \mid z - \boldsymbol{x} \in \mathcal{P}_{\boldsymbol{x}}, z \in B\}, |\rho_Q(\boldsymbol{T}\boldsymbol{x}, \boldsymbol{T}\boldsymbol{x}') - \rho_P(\boldsymbol{x}, \boldsymbol{x}')| \leq \delta,$$

where B denotes a ball given by the γ-cover.

Lemma 3.4. *Given a nonlinear neural network $\boldsymbol{T}$, if $||\boldsymbol{x}|| \leq b$ $\forall \boldsymbol{x} \in \mathcal{X}$, i.e., instances are norm bounded, then $\boldsymbol{T}$ is of γ-cover δ-isometry w.r.t. $\mathcal{X} - \mathcal{N}(\boldsymbol{T}_{|\boldsymbol{x}})$ of $\boldsymbol{x} \in S_m^{(x)}$, where δ and γ are respectively specified in Lemmas 3.1 and 3.3.*

See the proof in the supplementary material, available online.

Example 2. In a nonlinear DNN $\boldsymbol{T}$ as defined in Section 2.3, given an instance $\boldsymbol{x}$, it is δ-isometry w.r.t. variation subspace $\mathcal{X} - \mathcal{N}(T_{|\boldsymbol{x}})$. Note that for $\boldsymbol{x}$ of different instances, $T_{|\boldsymbol{x}}$ is potentially different. In practice, the singular values of weight matrices of the induced linear DNN $\boldsymbol{T}_{|\boldsymbol{x}}$ only relate to the variation in the space $\mathcal{X} - \mathcal{N}(T_{|\boldsymbol{x}})$. The bound given in the following will establish the relationship between generalization errors and singular values of weight matrices of DNNs by characterizing the constraints that DNNs impose on the variation in this space.

3.2 Main Results of Generalization Bound

With the local δ-isometry property of DNNs established, we derive in this section our main results of generalization bound.

Theorem 3.1. *Given a CRL problem, the algorithm to learn is a nonlinear neural network of L layers, denoted as $\boldsymbol{T}$. Suppose the following assumptions hold: 1) $||\boldsymbol{x}|| \leq b$ $\forall$ $\boldsymbol{x} \in \mathcal{X}$, i.e., instances are norm bounded; 2) the loss function $\mathcal{L}$ is bounded, a.k.a. $\forall z \in \mathcal{Z}, \mathcal{L}(f(\boldsymbol{x}), y) \leq M$, and the Lipschitz constant of $\mathcal{L} \circ f$ w.r.t $\boldsymbol{Tx}$ is bounded by A; 3) $\mathcal{X}$ is a regular k-dimensional manifold with a covering number $(\frac{C_{\mathcal{X}}}{\gamma/2})^k$; 4) within each covering ball B of $\mathcal{X}$ that contains $\boldsymbol{x} \in S_m^{(x)}$, $\boldsymbol{x} - \boldsymbol{x}' \in \mathcal{X} - \mathcal{N}(T_{|B})$ $\forall$ $\boldsymbol{x}, \boldsymbol{x}' \in B$. Then, for any $\nu > 0$, with probability at least $1 - \nu$ we have*

$$\mathrm{GE}(f_{S_m}) \leq A(\gamma + \delta') + M\sqrt{\frac{\log(2)2^{k+1}|\mathcal{Y}|C_{\mathcal{X}}^k}{\gamma^k m} + \frac{2\log(1/\nu)}{m}},$$

with

$$\delta' = 2b|\prod_{i=1}^{L}\sigma_{\max}^i - 1|, \gamma = \frac{o(S_m, \boldsymbol{T})}{\prod_{i=1}^{l(S_m,\boldsymbol{T})}\sigma_{\max}^i},$$

where $o(S_m, \boldsymbol{T})$ and $1 \leq l(S_m, \boldsymbol{T}) \leq L$ are values depending on the training set S_m and learned network $\boldsymbol{T}$.

A proof sketch is provided below, and the full proof is given in the supplementary material, available online.

Proof. By Lemma 3.4, we have that $\boldsymbol{T}$ is of γ-cover δ-isometry w.r.t. variation space of each training instance. The expansion property of δ-isometry gives $\rho_Q(\boldsymbol{Tx}, \boldsymbol{Tx}') \leq \rho_P(\boldsymbol{x}, \boldsymbol{x}') + 2b|\prod_{i=1}^{L}\sigma_{\max}^i - 1|$. By Theorem 2.2, we have that DNNs are $(|\mathcal{Y}|2^k C_{\mathcal{X}}^k/\gamma^k, A(\gamma + 2b|\prod_{i=1}^{L}\sigma_{\max}^i - 1|))$-robust. Note that the proof differs from Theorem 2.2 subtly, for that the loss difference needs to stay in the variation space of each covering ball. Since the tricky part is also present in the proof of Theorem 3.2 (the condition $\boldsymbol{x}' - \boldsymbol{x}_j \in \mathcal{P}_{\boldsymbol{x}_j}$ in Eq. (7)), to avoid tautology, we do not write the full proof here. The proof is finished by applying the robustness conclusion into Theorem 2.1. □

Regarding the 4th assumption in Theorem 3.1, it intuitively states that the local variation of interest w.r.t. each $\boldsymbol{x} \in S_m^{(x)}$ falls in the space $\mathcal{X} - \mathcal{N}(\boldsymbol{T}_{|B})$. Denote $\boldsymbol{v} = \boldsymbol{x} - \boldsymbol{x}'$, we have $\boldsymbol{Tv} = \boldsymbol{T}_{|B}\boldsymbol{v} = 0$ if $\boldsymbol{v} \in \mathcal{N}(\boldsymbol{T}_{|B})$, i.e., the variation vanishes after passing through the network. In practice, we are not interested in such a trivial case of vanishing local variations. Instead, it is the variation in the complement $\mathcal{X} - \mathcal{N}(\boldsymbol{T}_{|B})$ that we want to constrain.

We have assumed that $\mathcal{X}$ is a regular k-dimensional manifold, whose covering number is $(\frac{C_{\mathcal{X}}}{\gamma/2})^k$, where $C_{\mathcal{X}}$ is a constant that captures the "intrinsic" properties of $\mathcal{X}$, and γ is the diameter of the covering ball. Such an assumption is general enough to accommodate at least visual data such as natural images and has been widely used [53].

In Theorem 3.1, $|\mathcal{Y}|(\frac{C_{\mathcal{X}}}{\gamma/2})^k$ corresponds the covering number of the joint space $\mathcal{X} \times \mathcal{Y}$. We now show that with proper use of the contraction property of isometric mapping of DNNs, the constant $|\mathcal{Y}|$ in the second term of the upper bound can be removed while a modified first term has the potential to be small as well, indicating a better bound. Our idea is to directly exploit the covering number of the instance space $\mathcal{X}$ and measure the differences of the loss function $\mathcal{L}(f(\boldsymbol{Tx}), y)$ w.r.t. both arguments. Such a measure deals with instances of different labels but are close enough in $\mathcal{X}$, thus characterizing both the errors that are caused by erroneously contracting the distance between instances from different classes and erroneously expanding the distance between instances from the same classes, instead of those of the same classes alone. However, it will cause the issue of infinite Lipschitz constant of loss function $\mathcal{L}(f(\boldsymbol{x}), y)$. To see this, consider a binary classification problem whose loss function $\mathcal{L}$ is

$$\mathcal{L}(f(\boldsymbol{x}), y) = -\mathbf{1}_{y=1}\log f(\boldsymbol{x}) - \mathbf{1}_{y=0}\log(1 - f(\boldsymbol{x})),$$

where $(\boldsymbol{x}, y)$ is an example, $f(\boldsymbol{x})$ is a function that maps $\boldsymbol{x}$ to probability, and $\mathbf{1}$ is an indicator function. We provide an example case to illustrate the influence of metrics on $\mathcal{Z}$.

Case. Let the metric on $\mathcal{Z}$ be $\rho((\boldsymbol{x}, y), (\boldsymbol{x}', y')) = ||(\boldsymbol{x} - \boldsymbol{x}', y - y')|| = ||\boldsymbol{x} - \boldsymbol{x}'|| + |y - y'|$. Suppose that we have a pair of examples $(\boldsymbol{x}, y = 1)$ and $(\boldsymbol{x}, y' = 0)$ that only differ in labels, we have

$$A \geq \frac{|\mathcal{L}(f(\boldsymbol{x}), y) - \mathcal{L}(f(\boldsymbol{x}), y')|}{||(\boldsymbol{x}, y) - (\boldsymbol{x}, y')||} = \frac{\log(f(\boldsymbol{x})/(1 - f(\boldsymbol{x})))}{1}.$$

When there exists an $\boldsymbol{x}$ such that $f(\boldsymbol{x}) \to 1$, we have $|\mathcal{L}(f(\boldsymbol{x}), y) - \mathcal{L}(f(\boldsymbol{x}), y')| \to +\infty$. This happens because as y changes values, due to its discreteness, it could induce a jump discontinuity on $\mathcal{L}(f(\boldsymbol{x}), y)$, even though $\mathcal{L}(f(\boldsymbol{x}), y)$ is Lipschitz continuous w.r.t. $\boldsymbol{x}$. To avoid this, [59] and [27] employ a large covering number to ensure that examples in the same ball have the same label.

We note that derivation of generalization bounds for robust algorithms concerns with the loss difference $|\mathcal{L}(f(\boldsymbol{x}), y) - \mathcal{L}(f(\boldsymbol{x}'), y')|$ between example pairs. To address the aforementioned issue, we consider two separate cases for the loss difference: the cases of $y = y'$ and $y \neq y'$. For the case $y = y'$, we exploit the bounded Lipschitz constant of $\mathcal{L} \circ f$ w.r.t. $\boldsymbol{Tx}$. For the case $y \neq y'$, we introduce the following *pairwise error function* to characterize the loss difference.

Definition 13 (Pairwise error function). *Given a CRL problem, of which $\mathcal{L}$ is bounded for any compact set in $\mathcal{Z}$, a.k.a. for z in any compact subset of $\mathcal{Z}$, $\mathcal{L}(f(\boldsymbol{x}), y) \leq M$, $\mathcal{X}$ is a regular*

k-dimensional manifold with a γ-cover, and $\boldsymbol{T}$ is of γ-cover δ-isometry, a pairwise error function (PE) of the tuple $(\mathcal{L}, f, \boldsymbol{T}, \mathcal{Z}, \gamma)$ is defined as

$$\mathrm{PE}(\delta) = \max_{z=(\boldsymbol{x},y)\in\mathcal{Z}} \max_{z'\in D} |\mathcal{L}(f(\boldsymbol{T}\boldsymbol{x}), y) - \mathcal{L}(f(\boldsymbol{T}\boldsymbol{x}'), y')|$$

with $D = \{z' = (\boldsymbol{x}', y') \in \mathcal{Z} \mid \gamma - \delta \le ||\boldsymbol{T}\boldsymbol{x}' - \boldsymbol{T}\boldsymbol{x}|| \le \gamma + \delta, ||\boldsymbol{x} - \boldsymbol{x}'|| \le \gamma\}$.

It characterizes the largest loss difference for examples in $\mathcal{Z}$ that may arise due to the contraction and expansion properties of δ-isometry mapping. Note that $\mathrm{PE}(\delta)$ is a monotonously increasing function of δ — a larger δ means more feasible examples in $\mathcal{X}$ and possibly larger distance contraction/expansion, leading to a possibly larger value of $\mathrm{PE}(\delta)$.

Theorem 3.2. *Given a CRL problem, the algorithm to learn is a nonlinear neural network of L layers, denoted as $\boldsymbol{T}$. Suppose the following assumptions hold: 1) $||\boldsymbol{x}|| \le b \ \forall \ \boldsymbol{x} \in \mathcal{X}$, i.e., instances are norm bounded; 2) the loss function $\mathcal{L}$ is bounded, a.k.a. $\forall z \in \mathcal{Z}, \mathcal{L}(f(\boldsymbol{T}\boldsymbol{x}), y) \le M$, and the Lipschitz constant of $\mathcal{L} \circ f$ w.r.t $\boldsymbol{T}\boldsymbol{x}$ is bounded by A; 3) $\mathcal{X}$ is a regular k-dimensional manifold with a covering number $(\frac{C_{\mathcal{X}}}{\gamma/2})^k$; 4) within each covering ball B of $\mathcal{X}$ that contains $\boldsymbol{x} \in S_m^{(x)}$, $\boldsymbol{x} - \boldsymbol{x}' \in \mathcal{X} - \mathcal{N}(\boldsymbol{T}_{|B}) \ \forall \ \boldsymbol{x}, \boldsymbol{x}' \in B$. Then, for any $\nu > 0$, with probability at least $1 - \nu$ we have*

$$\mathrm{GE}(f_{S_m}) \le \max\{A(\gamma + \delta), \mathrm{PE}(\delta)\} \tag{5}$$

$$+M\sqrt{\frac{\log(2)2^{k+1}C_{\mathcal{X}}^k}{\gamma^k m} + \frac{2\log(1/\nu)}{m}}, \tag{6}$$

where $\delta = 2b\max(|\prod_{i=1}^L \sigma_{\max}^i - 1|, |\prod_{i=1}^L \sigma_{\min}^i - 1|)$, and γ is the same as that of Theorem 3.1.

Proof. Similar to the proof of Theorem 2.1, we partition the space $\mathcal{Z}$ via the assumed γ-cover. Since $\mathcal{X}$ is a k-dimensional manifold, its covering number is upper bounded by $C_{\mathcal{X}}^k/(\gamma/2)^k$. Let K be the overall number of covering set, which is upper bounded by $C_{\mathcal{X}}^k/(\gamma/2)^k$. Denote C_i the ith covering ball and let N_i be the set of indices of training examples that fall into C_i. Note that $(|N_i|)_{i=1,\ldots,K}$ is an IDD multimonial random variable with parameters m and $(|\mu(C_i)|)_{i=1,\ldots,K}$. Then

$$\begin{aligned}
&|R(f\boldsymbol{T}) - R_m(f\boldsymbol{T})| \\
&= \left|\sum_{i=1}^K \mathbb{E}_{z\sim\mu}[\mathcal{L}(f(\boldsymbol{T}\boldsymbol{x}), y)|z \in C_i]\mu(C_i) - \frac{1}{m}\sum_{i=1}^m \mathcal{L}(f(\boldsymbol{T}\boldsymbol{x}_i), y_i)\right| \\
&\le \left|\sum_{i=1}^K \mathbb{E}_{z\sim\mu}[\mathcal{L}(f(\boldsymbol{T}\boldsymbol{x}), y)|z \in C_i]\frac{|N_i|}{m} - \frac{1}{m}\sum_{i=1}^m \mathcal{L}(f(\boldsymbol{T}\boldsymbol{x}_i), y_i)\right| \\
&\quad + \left|\sum_{i=1}^K \mathbb{E}_{z\sim\mu}[\mathcal{L}(f(\boldsymbol{T}\boldsymbol{x}), y)|z \in C_i]\mu(C_i) \right. \\
&\quad \left. - \sum_{i=1}^K \mathbb{E}_{z\sim\mu}[\mathcal{L}(f(\boldsymbol{T}\boldsymbol{x}), y)|z \in C_i]\frac{|N_i|}{m}\right| \\
&\le \left|\frac{1}{m}\sum_{i=1}^K \sum_{j\in N_i} \max_{z'\in C_i, \boldsymbol{x}'-\boldsymbol{x}_j\in\mathcal{P}_{\boldsymbol{x}_j}} |\mathcal{L}(f(\boldsymbol{T}\boldsymbol{x}'), y') - \mathcal{L}(f(\boldsymbol{T}\boldsymbol{x}_j), y_j)|\right|
\end{aligned} \tag{7}$$

$$+\left|\max_{z\in\mathcal{Z}}\left|\mathcal{L}(f(\boldsymbol{T}\boldsymbol{x}), y)\right|\sum_{i=1}^K\left|\frac{|N_i|}{m} - \mu(C_i)\right|\right|. \tag{8}$$

Remember that $z = (\boldsymbol{x}, y)$. We consider the two cases of $y' = y_j$ and $y' \ne y_j$ to bound Eq. (7).

When $y' = y_j$, by the assumption that $\boldsymbol{T}$ is of γ-cover δ-isometry w.r.t. $\mathcal{P}_{\boldsymbol{x}}$ of $\boldsymbol{x} \in S_m^{(x)}$ and the Lipschitz constant of $\mathcal{L} \circ f$ w.r.t. $\boldsymbol{T}\boldsymbol{x}$ is A, suppose the maximum is achieved at $\boldsymbol{x}_k$ and $\boldsymbol{x}_k \in C_p$, we have

$$\begin{aligned}
&\max_{z'\in C_p, \boldsymbol{x}'-\boldsymbol{x}_k\in\mathcal{P}_{\boldsymbol{x}_k}} |\mathcal{L}(f(\boldsymbol{T}\boldsymbol{x}'), y') - \mathcal{L}(f(\boldsymbol{T}\boldsymbol{x}_k), y_k)| \\
&\quad \le A \max_{z'\in C_p, \boldsymbol{x}'-\boldsymbol{x}_k\in\mathcal{P}_{\boldsymbol{x}_k}} ||\boldsymbol{T}_{|\boldsymbol{x}_k}(\boldsymbol{x}' - \boldsymbol{x}_k)||
\end{aligned} \tag{9}$$

$$\begin{aligned}
&\le A \max_{z'\in C_p, \boldsymbol{x}'-\boldsymbol{x}_k\in\mathcal{P}_{\boldsymbol{x}_k}} \left(||\boldsymbol{x}' - \boldsymbol{x}_k|| + 2b\left|\prod_{i=1}^L \sigma_{\max}^i - 1\right|\right) \\
&\le A\left(\gamma + 2b\left|\prod_{i=1}^L \sigma_{\max}^i - 1\right|\right) \\
&\le A\left(\gamma + 2b\max\left(\left|\prod_{i=1}^L \sigma_{\max}^i - 1\right|, \left|\prod_{i=1}^L \sigma_{\min}^i - 1\right|\right)\right),
\end{aligned} \tag{10}$$

where the second inequality holds since the γ-cover δ-isometry of $\boldsymbol{T}$ also gives $\rho_Q(\boldsymbol{T}\boldsymbol{x}, \boldsymbol{T}\boldsymbol{x}') \le \rho_P(\boldsymbol{x}, \boldsymbol{x}') + 2b|\prod_{i=1}^L \sigma_{\max}^i - 1|$.

When $y' \ne y_j$, given any training $\boldsymbol{x}_k$, we have

$$\begin{aligned}
&\max_{z'\in C_p, \boldsymbol{x}'-\boldsymbol{x}_k\in\mathcal{P}_{\boldsymbol{x}_k}} |\mathcal{L}(f(\boldsymbol{T}\boldsymbol{x}'), y') - \mathcal{L}(f(\boldsymbol{T}\boldsymbol{x}_k), y_k)| \\
&\quad = \max_{z'\in D_{z_k}} |\mathcal{L}(f(\boldsymbol{T}\boldsymbol{x}'), y') - \mathcal{L}(f(\boldsymbol{T}\boldsymbol{x}_k), y_k)| \\
&\quad \le \mathrm{PE}\left(2b\max\left(\left|\prod_{i=1}^L \sigma_{\max}^i - 1\right|, \left|\prod_{i=1}^L \sigma_{\min}^i - 1\right|\right)\right),
\end{aligned} \tag{11}$$

where the inequality holds since by γ-cover δ-isometry of $\boldsymbol{T}$, we have $\rho_Q(\boldsymbol{T}\boldsymbol{x}, \boldsymbol{T}\boldsymbol{x}') \le \rho_P(\boldsymbol{x}, \boldsymbol{x}') + \delta$ and $\rho_Q(\boldsymbol{T}\boldsymbol{x}, \boldsymbol{T}\boldsymbol{x}') \ge \rho_P(\boldsymbol{x}, \boldsymbol{x}') - \delta$, with

$$\delta = 2b\max\left(\left|\prod_{i=1}^L \sigma_{\max}^i - 1\right|, \left|\prod_{i=1}^L \sigma_{\min}^i - 1\right|\right).$$

Thus Eq. (7) is less than or equal to $\max\{A(\gamma + \delta), \mathrm{PE}(\delta)\}$. By Breteganolle-Huber-Carol inequality, Eq. (8) is less than or equal to $M\sqrt{\frac{\log(2)2^{k+1}C_{\mathcal{X}}^k}{\gamma^k m} + \frac{2\log(1/\nu)}{m}}$.

The proof is finished. □

We now specify a case where the obtained bound in Theorem 3.2 is tighter than that in Theorem 3.1; for example, in the ball that covers $(\boldsymbol{x}_k, y_k)$, few examples with $y \ne y_k$ are misclassified. In this case, $\mathrm{PE}(2b\max(|\prod_{i=1}^L \sigma_{\max}^i - 1|, |\prod_{i=1}^L \sigma_{\min}^i - 1|) \le A(\gamma + 2b|\prod_{i=1}^L \sigma_{\max}^i - 1|)$, and the covering number is shrunken by a factor of $\sqrt{|\mathcal{Y}|}$. Our result only incrementally improves the generalization bound. However, it clearly shows that the contraction property of isometric mapping plays an important role in bounding the generalization error.

3.3 Suggestion of New Algorithms

Many quantities exist in the GE bound established in Theorem 3.2. Except γ and δ, all others are independent of the neural network. Although both γ and δ are controlled by singular values of weight matrices, we note that γ, which specifies the size of covering balls for a covering of $\mathcal{X}$, is more of a trade-off parameter that balances between the first and second term of the GE bound, than of a variable used to control GE, as long as its values satisfy the condition of $\gamma \leq o(S_m, \boldsymbol{T})/(\prod_{i=1}^{l(S_m,\boldsymbol{T})} \sigma_{\max}^i)$ established in Lemma 3.3.

To control the bound via δ, we note that the minimum value of the bound w.r.t. δ is achieved when $\delta = 0$, which implies $\prod_{i=1}^L \sigma_{\max}^i = 1$ and $\prod_{i=1}^L \sigma_{\min}^i = 1$. In the following lemma, we show that the condition is achieved only when $\sigma_{\max}^i = \sigma_{\min}^i, \forall i = 1, \ldots, L$.

Lemma 3.5. *In Theorem 3.2, $\delta = 0$ is achieved only when*

$$\sigma_{\max}^i = \sigma_{\min}^i, \ \forall i = 1, \ldots, L,$$

$$\prod_{i=1}^L \sigma_{\max}^i = 1, \prod_{i=1}^L \sigma_{\min}^i = 1.$$

Proof. It is straightforward to see that $\delta = 0$ i.f.f. $\prod_{i=1}^L \sigma_{\max}^i = 1$ and $\prod_{i=1}^L \sigma_{\min}^i = 1$. In the following, we show the two conditions hold only when $\sigma_{\max}^i = \sigma_{\min}^i$, $\forall i = 1, \ldots, L$.

For any $i \in \{1, \ldots, L\}$, we reparameterize $\sigma_{\min}^i$ as $\sigma_{\min}^i = \alpha_i \sigma_{\max}^i$. It is clear that $\alpha_i \in (0, 1]$.

Since $\prod_{i=1}^L \sigma_{\min}^i = \prod_{i=1}^L \sigma_{\max}^i = 1$, we have

$$1 = \prod_{i=1}^L \sigma_{\min}^i = \prod_{i=1}^L \alpha_i \sigma_{\max}^i = \prod_{i=1}^L \alpha_i. \tag{12}$$

Notice that $\alpha \in (0, 1]$, thus, we have $0 < \prod_{i=1}^L \alpha_i \leq 1$. To have Eq. (12), we need $\alpha_i = 1, \forall i = 1, \ldots, L$, indicating $\sigma_{\min}^i = \sigma_{\max}^i, \forall i = 1, \ldots, L$. □

We show in Lemma 3.5 that the optimal GE bound of Theorem 3.2 w.r.t. δ is achieved only when all singular values of each of weight matrices of a DNN are equal. Among various solutions, the most straightforward one is that all singular values are equal to 1; in other words, each weight matrix has orthonormal rows or columns. This inspires a new set of algorithms that we generally term as *Orthogonal Deep Neural Networks (OrthDNNs)*.

4 Algorithms of Orthogonal Deep Neural Networks

In this section, we first present the algorithm of *strict OrthDNNs* by enforcing strict orthogonality of weight matrices during network training. It amounts to optimizing weight matrices on their respective Stiefel manifolds, which however, is computationally prohibitive for large-sized networks. To achieve efficient OrthDNNs, we propose a novel algorithm called Singular Value Bounding (SVB), which achieves *approximate OrthDNNs* via a simple scheme of hard regularization. We discuss alternative schemes of soft regularization for approximate OrthDNNs, and compare with our proposed SVB. Batch Normalization [28] is commonly used to accelerate training of modern DNNs, yet it has a potential risk of ill-conditioned layer transform, causing its incompatibility with OrthDNNs. In fact, direct use of BN in OrthDNNs makes it ineffective to enforce strict orthogonality of weight matrices. We propose Degenerate Batch Normalization (DBN) to enable its use with strict OrthDNNs. We also propose Bounded Batch Normalization (BBN) to remove the potential risk of ill-conditioned layer transform. We finally explain how OrthDNNs are used for convolutional kernels.

Denote parameters of a DNN collectively as $\Theta = \{\mathbf{W}_l, \mathbf{b}_l\}_{l=1}^L$, where $\{b_l\}_{l=1}^L$ are bias terms. We discuss algorithms of strict or approximate OrthDNNs in the following context. Given a training set $\{\mathbf{x}_i, y_i\}_{i=1}^m$, we write the training objective as $\mathcal{L}\big(\{\mathbf{x}_i, y_i\}_{i=1}^m; \Theta\big)$. Training is based on SGD (or its variants [51]), which updates Θ via a simple rule of $\Theta^{t+1} \leftarrow \Theta^t - \eta \frac{\partial \mathcal{L}}{\partial \Theta^t}$, where η is the learning rate, and the gradient $\frac{\partial \mathcal{L}}{\partial \Theta^t}$ is usually computed from a mini-batch of training examples. Network training proceeds by sampling for each iteration t a mini-batch from $\{\mathbf{x}_i, y_i\}_{i=1}^m$, until a specified number of iterations or the training loss plateaus.

4.1 The Case of Strict Orthogonality

Enforcing orthogonality of weight matrices during network training amounts to solving the following constrained optimization problem

$$\begin{aligned} &\min_{\Theta = \{\mathbf{W}_l, \mathbf{b}_l\}_{l=1}^L} \mathcal{L}\big(\{\mathbf{x}_i, y_i\}_{i=1}^m; \Theta\big) \\ &\text{s.t. } \mathbf{W}_l \in \mathcal{O} \ \forall \ l \in \{1, \ldots, L\}, \end{aligned} \tag{13}$$

where $\mathcal{O}$ stands for the set of matrices whose row or column vectors are orthonormal. For $\mathbf{W}_l$ of any lth layer, problem (13) in fact constrains its solution set as a Riemannian manifold called Stiefel manifold, which is defined as $\mathcal{M}_l = \{\mathbf{W}_l \in \mathbb{R}^{n_l \times n_{l-1}} | \mathbf{W}_l^\top \mathbf{W}_l = \mathbf{I}\}$ assuming $n_l \geq n_{l-1}$, and is an embedded submanifold of the space $\mathbb{R}^{n_l \times n_{l-1}}$, where $\mathbf{I}$ is an identity matrix. In literature, optimization of a differentiable cost function on such a matrix manifold and its convergence analysis have been intensively studied [1], [9]. For completeness, we briefly present the solving algorithm of (13) as follows.

Denote $T_{\mathbf{W}_l}\mathcal{M}_l$ as the tangent space to $\mathcal{M}_l$ at the current $\mathbf{W}_l \in \mathcal{M}_l$. First-order methods such as SGD first find a tangent vector $\Omega_{\mathbf{W}_l} \in T_{\mathbf{W}_l}\mathcal{M}_l$ that describes the steepest descent direction for the cost, and update $\mathbf{W}_l$ as $\mathbf{W}_l - \eta\Omega_{\mathbf{W}_l}$ with the step size η that satisfies conditions of convergence, and then perform a retraction $\mathcal{R}_{\mathbf{W}_l}(-\eta\Omega_{\mathbf{W}_l})$ that defines a mapping from the tangent space to the Stiefel manifold, which can be achieved by $\mathcal{R}_{\mathbf{W}_l}(-\eta\Omega_{\mathbf{W}_l}) = \mathcal{Q}(\mathbf{W}_l - \eta\Omega_{\mathbf{W}_l})$, where the operator $\mathcal{Q}$ denotes the Q factor of QR matrix decomposition. To obtain the tangent vector $\Omega_{\mathbf{W}_l}$, one may project the gradient $\frac{\partial \mathcal{L}}{\partial \mathbf{W}_l}$ in the embedding space $\mathbb{R}^{n_l \times n_{l-1}}$ (or its momentum version [51]) onto the tangent space $T_{\mathbf{W}_l}\mathcal{M}_l$ by $\mathcal{P}_{\mathbf{W}_l}\frac{\partial \mathcal{L}}{\partial \mathbf{W}_l}$, where $\mathcal{P}_{\mathbf{W}_l}$ defines a projection operator according to the local geometry of $\mathbf{W}_l \in \mathcal{M}_l$. Convergence analysis for such a scheme to obtain the tangent vector is presented in [44]. In the supplementary material, available online, we present the algorithmic details for optimization of weight matrices on the Stiefel manifolds.

4.2 Achieving Near Orthogonality via Singular Value Bounding

Constraining solutions of weight matrices of a DNN on their Stiefel manifolds is an interesting direction of research. It also supports analysis of theoretical properties as in Section 3 and the related works [44], [47]. However, it arguably has the following shortcomings concerned with computation, empirical performance, and also compatibility with existing deep learning methods, which motivate us to address these shortcomings by developing new algorithms of approximate OrthDNNs.

- Strict constraining of weight matrices on the Stiefel manifolds requires expensive computations — in particular, the operations of projecting the euclidean gradient onto the tangent space and retraction onto the Stiefel manifold dominate the costs in each iteration. If we allow the solutions slightly away from the manifolds, the expensive projection and retraction operations are not necessary to be performed in each iteration. Instead, similar pulling-back operations can be performed less frequently, e.g., in every a certain number of iterations, and consequently such a burden of pulling back is amortized.
- Theorem 3.2 gives a bound $GE(f_{S_m})$ of the expected error $R(f_{S_m})$ w.r.t the training error $R_m(f_{S_m})$. To achieve good performance on practical problems, both $R_m(f_{S_m})$ and $GE(f_{S_m})$ should be small. However, optimization of DNNs is characterized by proliferation of local optima/critical points [13], [30]. When we are motivated to optimize weight matrices on their Stiefel manifolds, obtaining $\mathbf{W}_l \in \mathcal{M}_l,\ l \in \{1, \ldots, L\}$, with $\Omega_{\mathbf{W}_l} = 0$, it is very likely that for a $\mathbf{W}_l$, there exists a better local optimum in the embedding euclidean space that is slightly away from the manifold (e.g., $\Omega_{\mathbf{W}_l} = 0$ while $\frac{\partial \mathcal{L}}{\partial \mathbf{W}_l} \neq 0$, or the euclidean gradient is in the complement null space of the current tangent space), and has a smaller $R_m(f_{S_m})$. If we allow the optimization to step away from, but still pivot around, the manifold, better solutions could be obtained by escaping from local optima on the manifold.
- Successful training of modern DNNs depends heavily on BN [28], a technique that can greatly improve training convergence and empirical results. However, as analyzed shortly in Section 4.4, BN would change the spectrum of singular values of each layer transform (i.e., the combined linear transform of each layer achieved by weight mapping and BN, as specified in (17)). Consequently, the efforts spending on enforcing strict orthogonality of weight matrices become ineffectual. Algorithms of approximate OrthDNNs seem more compatible with BN transform.

To develop an algorithm of approximate OrthDNNs, we propose a simple yet effective network training method called Singular Value Bounding (SVB). SVB is a sort of projected SGD method and can be summarized as follows: SVB simply bounds, after every T_{svb} iterations of SGD training, all the singular values of each $\mathbf{W}_l$, for $l = 1, \ldots, L$, in a narrow band $[1/(1+\epsilon), (1+\epsilon)]$ around the value of 1, where $\epsilon \geq 0$ is a specified small constant. Algorithm 1 presents the details.

Algorithm 1. Singular Value Bounding

input: A network of L layers with trainable parameters $\Theta = \{\mathbf{W}_l, \mathbf{b}_l\}_{l=1}^{L}$, training loss $\mathcal{L}$, learning rate η, the maximal number T of training iterations, a specified number T_{svb} of iteration steps, a small constant ϵ

Initialize Θ such that $\mathbf{W}_l^\top \mathbf{W}_l = \mathbf{I}$ or $\mathbf{W}_l \mathbf{W}_l^\top = \mathbf{I}$ for $l = 1, \ldots, L$
for $t = 0, \ldots, T-1$ **do**
  Update $\Theta^{t+1} \leftarrow \Theta^t - \eta \frac{\partial \mathcal{L}}{\partial \Theta^t}$ using SGD based methods
  while *training proceeds for every* T_{svb} *iterations* **do**
    for $l = 1, \ldots, L$ **do**
      Perform $[\mathbf{U}_l, \mathbf{\Sigma}_l, \mathbf{V}_l] = \mathrm{svd}(\mathbf{W}_l)$
      Let $\{\sigma_i^l\}_{i=1}^{n_l}$ be the diagonal entries of $\mathbf{\Sigma}_l$
      for $i = 1, \ldots, n_l$ **do**
        $\sigma_i^l = 1 + \epsilon$ if $\sigma_i^l > 1 + \epsilon$
        $\sigma_i^l = 1/(1+\epsilon)$ if $\sigma_i^l < 1/(1+\epsilon)$
      end
      Update $\mathbf{W}_l \leftarrow \mathbf{U}_l \mathbf{\Sigma}_l \mathbf{V}_l^\top$ with the bounded diagonal entries $\{\sigma_i^l\}_{i=1}^{n_l}$ of $\mathbf{\Sigma}_l$
    end
  end
end

output: Trained network with parameters Θ^T for inference

After each bounding step, optimization of SVB in fact proceeds in the embedding euclidean space, to search for potentially better solutions, before next bounding step that pulls the solutions back onto ($\epsilon = 0$) or near ($\epsilon > 0$) the Stiefel manifolds. With annealed learning rate schedules, we observe empirical convergence of SVB. Compared with manifold optimization in Section 4.1, SVB is more efficient since the dominating computation of SVD is invoked only every a certain number of iterations. Experiments of image classification in Section 5 show that SVB sometimes outperforms the algorithm of strict OrthDNNs in Section 4.1, both of which outperform the commonly used SGD based methods, and in many cases with a large margin.

4.3 Alternative Algorithms for Approximate OrthDNNs

To achieve approximate OrthDNNs, one may alternatively penalize the main objective $\mathcal{L}\big(\{\mathbf{x}_i, y_i\}_{i=1}^m; \Theta\big)$ with an augmented term that encourages orthonormality of columns or rows of weight matrices, resulting in the following unconstrained optimization problem

$$\min_{\Theta = \{\mathbf{W}_l, \mathbf{b}_l\}_{l=1}^L} \mathcal{L}\big(\{\mathbf{x}_i, y_i\}_{i=1}^m; \Theta\big) + \lambda \sum_{l=1}^{L} \|\mathbf{W}_l^\top \mathbf{W}_l - \mathbf{I}\|_F^2, \tag{14}$$

where $\|\cdot\|_F$ denotes Frobenius norm, λ is the penalty parameter, and we have assumed $n_l \geq n_{l-1}$ for a certain layer l. By using increasingly larger values of λ, the problem (14) approaches to achieve strict OrthDNNs. One can use SGD based methods to solve (14), where the additional computation cost incurred by the regularizer is marginal. Soft regularization of the type (14) is used in the related works [12], [57].

To relax the requirement of $n_l \geq n_{l-1}$ assumed in (14), an algorithm termed Spectral Restricted Isometry Property (SRIP) regularization, which leverages the matrix RIP condition [11], is proposed in [5], whose objective is written as

$$\min_{\Theta = \{\mathbf{W}_l, \mathbf{b}_l\}_{l=1}^L} \mathcal{L}\big(\{\mathbf{x}_i, y_i\}_{i=1}^m; \Theta\big) + \kappa \sum_{l=1}^{L} \sigma_{\max}(\mathbf{W}_l^\top \mathbf{W}_l - \mathbf{I}), \tag{15}$$

where κ is a penalty parameter, and $\sigma_{\max}(\cdot)$ denotes the spectral norm of a matrix. Although computation of (15) involves expensive eigen-decomposition, it can be efficiently approximated via power iteration method. One may refer to [5] for the solving equation. In this work, we compare the alternative (14) and (15) with our proposed SVB.

4.4 Compatibility with Batch Normalization

We start this section by showing that the original design of Batch Normalization [28] is incompatible with our proposed OrthDNNs. Technically, for a network layer that computes, before the nonlinear activation, $\mathbf{h} = \mathbf{W}\mathbf{x} \in \mathbb{R}^n$, BN inserts a normalization denoted as $\mathrm{BN}(\mathbf{h}) = \mathrm{BN}(\mathbf{W}\mathbf{x})$, where we have ignored the bias term for simplicity. BN in fact applies the following linear transformation to $\mathbf{h}$

$$\mathrm{BN}(\mathbf{h}) = \boldsymbol{\Upsilon}\boldsymbol{\Phi}(\mathbf{h} - \boldsymbol{\mu}) + \boldsymbol{\beta}, \tag{16}$$

where each entry of $\boldsymbol{\mu} \in \mathbb{R}^n$ is the output mean at each of the n neurons of the layer, the diagonal matrix $\boldsymbol{\Phi} \in \mathbb{R}^{n\times n}$ contains entries $\{1/\phi_i\}_{i=1}^n$ that is the inverse of the neuron-wise output standard deviation ϕ_i (obtained by adding a small constant to the variance for numerical stability), $\boldsymbol{\Upsilon} \in \mathbb{R}^{n\times n}$ is a diagonal matrix containing trainable scalar parameters $\{\upsilon_i\}_{i=1}^n$, and $\boldsymbol{\beta} \in \mathbb{R}^n$ is a trainable bias term. Note that during training, μ and ϕ for each neuron are computed using mini-batch examples, and during inference they are fixed representing the statistics of all the training population, which are usually obtained by running average. Thus the computation (16) for each example is deterministic after network training.

Inserting $\mathbf{h} = \mathbf{W}\mathbf{x}$ into (16) we get

$$\mathrm{BN}(\mathbf{x}) = \widetilde{\mathbf{W}}\mathbf{x} + \tilde{\mathbf{b}} \ \text{ s.t. } \ \widetilde{\mathbf{W}} = \boldsymbol{\Upsilon}\boldsymbol{\Phi}\mathbf{W} \ \ \tilde{\mathbf{b}} = \boldsymbol{\beta} - \boldsymbol{\Upsilon}\boldsymbol{\Phi}\boldsymbol{\mu}, \tag{17}$$

which is simply a standard layer with change of variables. The following lemma suggests that BN is incompatible with OrthDNNs: even though $\mathbf{W}$ is enforced to have orthonormal rows or columns, BN would change the conditioning of layer transform, i.e., spectrum of singular values of $\widetilde{\mathbf{W}}$, by learning $\boldsymbol{\Upsilon}$ and $\boldsymbol{\Phi}$ whose product does not necessarily contain diagonal entries $\{\upsilon_i/\phi_i\}_{i=1}^n$ of equal value.

Lemma 4.1. *For a matrix $\mathbf{W} \in \mathbb{R}^{M\times N}$ with singular values of all 1, and a diagonal matrix $\mathbf{G} \in \mathbb{R}^{M\times M}$ with nonzero entries $\{g_i\}_{i=1}^M$, let $g_{\max} = \max(|g_1|, \ldots, |g_M|)$ and $g_{\min} = \min(|g_1|, \ldots, |g_M|)$, the singular values of $\widetilde{\mathbf{W}} = \mathbf{G}\mathbf{W}$ is bounded in $[g_{min}, g_{\max}]$. When $\mathbf{W}$ is fat, i.e., $M \leq N$, and $\mathrm{rank}(\mathbf{W}) = M$, singular values of $\widetilde{\mathbf{W}}$ are exactly $\{|g_i|\}_{i=1}^M$.*

See the proof in the supplementary material, available online.

To make BN compatible with *strict OrthDNNs*, we propose *Degenerate Batch Normalization (DBN)* that learns layer-wise $\bar{\upsilon}$ and $\bar{\phi}$ instead of neuron-wise $\{\upsilon_i\}_{i=1}^n$ and $\{\phi_i\}_{i=1}^n$, so that the learned $\boldsymbol{\Upsilon}$ and $\boldsymbol{\Phi}$ respectively contain diagonal entries of equal value. Such a $\boldsymbol{\Upsilon}$ is learned simply by using a single trainable parameter $\bar{\upsilon}$ shared by all n neurons of the layer. To learn such a $\boldsymbol{\Phi}$, DBN still computes neuron-wise $\{\phi_i\}_{i=1}^n$ in each iteration of training, but uses running average to learn $\bar{\phi} = \frac{1}{n}\sum_{i=1}^n \phi_i$ that would be shared by the n neurons. The proposed DBN enjoys the benefit of neuron-wise normalization in BN, and when training converges, it learns a layer transform $\widetilde{\mathbf{W}} = \boldsymbol{\Upsilon}\boldsymbol{\Phi}\mathbf{W} = \bar{\upsilon}/\bar{\phi}\mathbf{W}$ whose conditioning is the same as that of $\mathbf{W}$, thus achieving compatibility with strict OrthDNNs.

Section 4.2 discusses the potential advantages of approximate OrthDNNs over the strict ones. To make BN compatible with *approximate OrthDNNs*, especially with our proposed SVB method, we propose *Bounded Batch Normalization (BBN)* that controls the variations among $\{\upsilon_i/\phi_i\}_{i=1}^n$, so that the conditioning of layer transform is not severely affected by $\boldsymbol{\Upsilon}\boldsymbol{\Phi}$. More specifically, BBN computes $\alpha = \frac{1}{n}\sum_{i=1}^n \upsilon_i/\phi_i$ in each iteration of training, and bounds each of $\{\frac{1}{\alpha}\upsilon_i/\phi_i\}_{i=1}^n$ in a narrow band $[1/(1+\tilde{\epsilon}), (1+\tilde{\epsilon})]$ around the value of 1, where $\tilde{\epsilon} \geq 0$ is a scalar parameter. Algorithm 2 presents details of the proposed BBN.

Algorithm 2. Bounded Batch Normalization

input: A network with L BN layers, trainable parameters $\{\boldsymbol{\Upsilon}_l^t\}_{l=1}^L$, $\{\boldsymbol{\beta}_l^t\}_{l=1}^L$, and statistics $\{\boldsymbol{\mu}_l^t\}_{l=1}^L$, $\{\boldsymbol{\Phi}_l^t\}_{l=1}^L$ of BN layers at iteration t, a small constant $\tilde{\epsilon}$

Update to get $\{\boldsymbol{\Upsilon}_l^{t+1}\}_{l=1}^L$ from $\{\boldsymbol{\Upsilon}_l^t\}_{l=1}^L$ (and $\{\boldsymbol{\beta}_l^{t+1}\}_{l=1}^L$ from $\{\boldsymbol{\beta}_l^t\}_{l=1}^L$), using SGD based methods
Update to get $\{\boldsymbol{\Phi}_l^{t+1}\}_{l=1}^L$ from $\{\boldsymbol{\Phi}_l^t\}_{l=1}^L$ (and $\{\boldsymbol{\mu}_l^{t+1}\}_{l=1}^L$ from $\{\boldsymbol{\mu}_l^t\}_{l=1}^L$), using running average over statistics of mini-batch examples
for $l = 1, \ldots, L$ **do**
  Let $\{\upsilon_i\}_{i=1}^{n_l}$ and $\{1/\phi_i\}_{i=1}^{n_l}$ be respectively the diagonal entries of $\boldsymbol{\Upsilon}_l^{t+1}$ and $\boldsymbol{\Phi}_l^{t+1}$
  Let $\alpha = \frac{1}{n_l}\sum_{i=1}^{n_l} \upsilon_i/\phi_i$
  for $i = 1, \ldots, n_l$ **do**
    $\upsilon_i = \alpha\phi_i(1+\tilde{\epsilon})$ if $\frac{1}{\alpha}\upsilon_i/\phi_i > 1+\tilde{\epsilon}$
    $\upsilon_i = \alpha\phi_i/(1+\tilde{\epsilon})$ if $\frac{1}{\alpha}\upsilon_i/\phi_i < 1/(1+\tilde{\epsilon})$
  end
end

output: Updated BN parameters and statistics at iteration $t+1$

The introduction of DBN makes it possible to empirically compare strict and approximate OrthDNNs in the context of modern architectures, which is presented in Section 5. Experiments in Section 5 also show that performance is improved when using BBN instead of BN, confirming the benefit by resolving BN's compatibility with OrthDNNs.

4.5 Orthogonal Convolutional Neural Networks

In previous sections, we present theories and algorithms of OrthDNNs by writing their layer-wise weights in matrix forms. When applying DNNs to image data, one is actually using networks with convolutional layers. For an lth convolutional layer with weight tensor of the size $n_l \times n_{l-1} \times n_h \times n_w$, where n_h and n_w denote the height and width of the convolutional kernel, we choose to convert the tensor as a matrix of the size $n_l \times n_{l-1}n_hn_w$ based on the following rational. Natural images are usually modeled by first learning filters from (densely overlapped) local patches, and then applying the thus learned filters to images to aggregate the corresponding local statistics. The convolutional layer in fact linearly transforms the n_{l-1} input feature maps in the same way, by applying each of n_l filters of the size $n_{l-1} \times n_h \times n_w$ to n_{l-1} patches of the size $n_h \times n_w$ in a sliding window fashion, resulting in local responses that are arrayed in the form of n_l feature

maps, which have the same size as that of input feature maps when padding the boundaries. In other words, the convolutional layer applies linear transformation, using n_l filters, to $n_{l-1}n_h n_w$-dimensional instances that are collected from local patches of input feature maps. Correspondingly, we choose to convert the weight tensor containing the n_l filters of dimension $n_{l-1}n_h n_w$ to its matrix form, and apply to it specific algorithms of OrthDNNs.

However, we note that our way of forming the weight matrices does not exactly specify linear transformations of convolutional layers. For the lth layer, its exact weight matrix is in fact a function of the kernel tensor and contains doubly block circulant submatrices [48]. Our preliminary experiments show that OrthDNNs based on such forms are effective to regularize network training as well. We would conduct further investigations with additional experiments in future research.

5 Experiments

We present in this section extensive experiments of image classification to verify the efficacy of OrthDNNs. We are particularly interested in how algorithms of strict or approximate OrthDNNs provide regularization to various architectures of modern DNNs, such as ConvNets [37], [49], ResNets [21], [22], DenseNet [26], and ResNeXt [58]. We use the benchmark datasets of CIFAR10, CIFAR100 [35], and ImageNet [46] for these experiments. We compare empirical performance and efficiency among different algorithms of strict and approximate OrthDNNs. For some of these comparisons, we also investigate behaviours of OrthDNNs under regimes of both small and large sizes of training samples, and robustness of OrthDNNs against corruptions that are commonly encountered in natural images, in order to better understand the empirical strength of OrthDNNs. For network training, we use SGD with momentum and initialize networks using orthogonal weight matrices, where the momentum is set as 0.9 with a weight decay of 0.0001. When our proposed SVB is turned on, we apply it to weight matrices of all layers after every epoch of training.

5.1 Comparative Studies on Algorithms of Strict and Approximate OrthDNNs

In this section, we use architectures of ConvNet and ResNet on CIFAR10 to study the behaviours of strict and approximate OrthDNNs. The CIFAR10 dataset consists of 60,000 32×32 color images of 10 object categories (50,000 training and 10,000 testing ones). We use raw images without pre-processing. Data augmentation follows the standard manner in [39]: during training, we zero-pad 4 pixels along each image side, and sample a 32×32 region crop from the padded image or its horizontal flip; during testing, we use the original non-padded image. Our ConvNet architectures follow [21], [49]. Each network starts with a conv layer of 16 3×3 filters, and then sequentially stacks three types of $2X$ conv layers of 3×3 filters, each of which has the feature map sizes of 32, 16, and 8, and filter numbers of 16, 32, and 64, respectively; spatial subsampling of feature maps is achieved by conv layers of stride 2; the network ends with a global average pooling and a fully-connected layer. The ResNet construction is based on the ConvNets presented above, where we use an "identity shortcut" to connect every two conv layers of 3×3 filters and use a "projection shortcut" when sub-sampling of feature maps is needed; we adopt the pre-activation version [22]. Thus, for both types of networks, we have $6X + 2$ weight layers in total. We set $X = 3$ for experiments in this section, giving networks of 20 weight layers.

TABLE 1
Comparison of Strict and Approximate OrthDNNs on the CIFAR10 Dataset [35], Using ConvNet and ResNet Architectures Respectively of 20 Weight Layers (Referring to the Main Text for Their Specifics)

Network	Training method	Error rate (%)	Averaged time per iter. (sec.)
ConvNet	SGD with momentum	16.68	0.0702
	Strict OrthDNNs via Manifold Opt.	10.85	0.2034
	Approx. OrthDNNs via Soft Regu.	10.39	0.0930
	Approx. OrthDNNs via SRIP	10.67	0.0760
	Approx. OrthDNNs via SVB	11.41	0.0718
ResNet	SGD with momentum	10.27	0.0754
	Strict OrthDNNs via Manifold Opt.	9.03	0.2042
	Approx. OrthDNNs via Soft Regu.	8.91	0.0844
	Approx. OrthDNNs via SRIP	8.72	0.0832
	Approx. OrthDNNs via SVB	8.76	0.0768

Degenerate batch normalization is used due to its compatibility with strict OrthDNNs.

The DBN proposed in Section 4.4 is designed to be compatible with strict OrthDNNs. We use DBN to enable training and comparison of strict and approximate OrthDNNs on the networks constructed above. We implement strict OrthDNNs as the algorithm presented in Section 4.1. We use our proposed SVB and those in [5], [12], [57] (i.e., the problems (14) and (15)) for approximate OrthDNNs. The learning rates start at 0.1 and end at 0.001, and decay every two epochs until the end of 160 epochs of training, where we set the mini-batch size as 128. We fix the parameter ϵ of SVB as 0.05, while both λ of soft regularization in (14) and κ of SRIP in (15) are optimally tuned as 0.1.

Table 1 gives the results with the curves of training convergence plotted in Fig. 3. Table 1 shows that on both of the two networks, algorithms of strict and approximate OrthDNNs outperform standard SGD based method, confirming the improved generalization by their regularization of network training. Moreover, approximate OrthDNNs via either SVB, soft regularization, or SRIP perform as well as strict ones, but at a much lower computational cost,[2] suggesting their advantage in practical use. Due to the prohibitive computation of strict OrthDNNs on modern architectures of larger sizes, we choose to use approximate OrthDNNs in subsequent experiments, and correspondingly use BN or our proposed BBN to replace DBN.

2. In Table 1, the respective dominating computations of QR decomposition for manifold optimization and singular value decomposition for SVB are based on CUDA implementation.

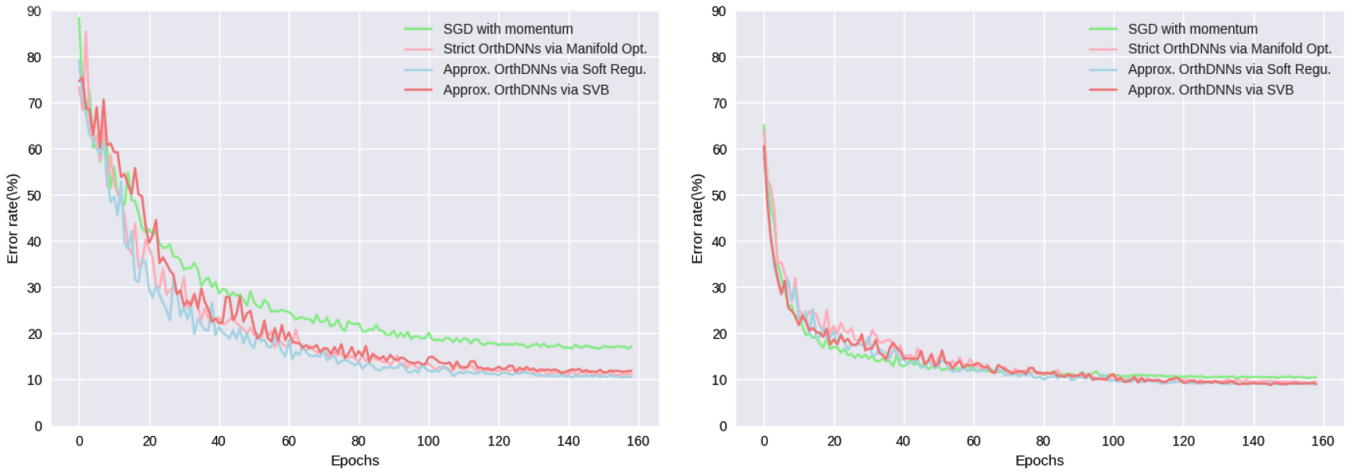


Fig. 3. Validation curves of strict and approximate OrthDNNs on the CIFAR10 dataset [35] using architectures of a ConvNet (left) and a ResNet (right) respectively of 20 weight layers.

5.2 Comparison of Hard and Soft Regularization for Approximate OrthDNNs

In this section, we study algorithms of approximate OrthDNNs by comparing our proposed SVB with soft regularization [12], [57] and SRIP [5]. The experiments are conducted on CIFAR10 using a pre-activation version of ResNet constructed in the same way as in Section 5.1. Setting $X = 11$ gives a total of 68 weight layers. To train the network, we use learning rates that start at 0.5 and end at 0.001, and decay every two epochs until the end of 160 epochs of training, where we set the mini-batch size as 128. Comparison is made with the baseline of standard SGD with momentum. We also switch BBN on or off to verify its effectiveness. We fix ϵ of SVB as 0.5, while the penalty λ of soft regularization and κ of SRIP are optimally tuned as 0.005 and 0.01 respectively. We fix $\tilde{\epsilon}$ of BBN as 0.2. We run each setting of experiments for five times, and report results in the format of best (mean $\pm$ standard deviation).

Table 2 shows that approximate OrthDNNs via SVB, soft regularization, and SRIP provide effective regularization to network training, and SVB and SRIP outperform soft regularization with a noticeable margin. Compared with BN, our proposed BBN can better regularize training and give slightly improved performance. Note that algorithmic design of BBN may not be compatible with soft regularization and RRIP, which explains the degraded performance when using them together.

TABLE 2
Comparison of Approximate OrthDNNs via Hard and Soft Regularization on the CIFAR10 Dataset [35], Using a ResNet of 68 Weight Layers

Training method	Error rate (%)
SGD with momentum + BN	6.25 (6.43 ± 0.15)
Soft Regularization + BN	6.12 (6.28 ± 0.12)
SRIP + BN	5.86 (5.95 ± 0.08)
SVB + BN	5.84 (5.96 ± 0.17)
Soft Regularization + BBN	6.22 (6.30 ± 0.07)
SRIP + BBN	5.99 (6.10 ± 0.11)
SVB + BBN	5.79 (5.88 ± 0.07)

Each setting is run for five times, and results are reported in the format of best (mean $\pm$ standard deviation)

5.3 Experiments With Modern Architectures

In this section, we investigate how our proposed SVB and BBN methods provide regularization to modern architectures of ResNet [22], Wide ResNet [61], DenseNet [26], and ResNeXt [58]. We use CIFAR10, CIFAR100 [35], and ImageNet [46] for these experiments. The CIFAR100 dataset has the same number of 32×32 color images as CIFAR10 does, but it has 100 object categories where each category contains one-tenth images of those of CIFAR10. We use data augmentation in the same way as for CIFAR10. The ImageNet dataset contains 1.28 million images of 1,000 categories for training, and 50,000 images for validation. We use data augmentation as in [58].

For experiments on CIFAR10 and CIFAR100, we use the following specific architectures. ResNet is constructed in the same way as in Section 5.1; we set $X = 9$ here giving a total of 56 weight layers. Wide ResNet is the same as "WRN-28-10" in [61]. ResNeXt is the same as "ResNeXt-29 (16×64d)" in [58], i.e., the depth $L = 29$, cardinality $C = 16$, and the feature width in each cardinal branch $d = 64$. We use consistent hyper-parameters to train these architectures. The learning rates start at 0.5 and end at 0.001, and decay every two epochs until the end of 300 epochs of training, where we set the mini-batch size as 64 — note that this schedule with more training epochs and smaller mini-batch size usually gives better empirical performance than the training schedule used in Section 5.2 does. We fix ϵ and $\tilde{\epsilon}$ of SVB and BBN as 0.5 and 0.2 respectively. Table 3 confirms that SVB and BBN improve generalization of various architectures. We also observe that

TABLE 3
Error Rates (%) on the CIFAR10 and CIFAR100 [35] Datasets When Applying our Proposed SVB and BBN to Various Modern Architectures (Referring to the Main Text for Their Specifics)

Method	CIFAR10	CIFAR100
ResNet W/O SVB+BBN	5.68	27.71
ResNet WITH SVB+BBN	5.28	26.47
Wide ResNet W/O SVB+BBN	3.78	20.02
Wide ResNet WITH SVB+BBN	3.24	18.75
ResNeXt W/O SVB+BBN	4.12	20.65
ResNeXt WITH SVB+BBN	3.33	16.94

TABLE 4
Error Rates (%) on the Validation Set of ImageNet [46] When Applying Our Proposed SVB and BBN to Various Modern Architectures (Referring to the Main Text for Their Specifics)

Method	Top-1 error	Top-5 error
ResNet W/O SVB+BBN	21.00	5.72
ResNet WITH SVB+BBN	20.74	5.35
DenseNet W/O SVB+BBN	22.32	6.33
DenseNet WITH SVB+BBN	21.80	5.83
ResNeXt W/O SVB+BBN	19.39	4.43
ResNeXt WITH SVB+BBN	18.89	4.27

Results are based on single-crop testing of the size 320×320.

improvements on CIFAR100 are generally greater than those on CIFAR10, which may be due to the problem nature of smaller sample size for CIFAR100. We will investigate how our methods perform with varying sample sizes more thoroughly in the subsequent section.

For experiments on ImageNet, we use top-performing models of the following architectures: "ResNet-152" of [22], "DenseNet-264" of [26], and "ResNeXt-101 (64 $\times$4 d)" of [58]. To train these models, we use the same hyper-parameters as respectively reported in these methods. The parameters ϵ and $\tilde{\epsilon}$ of SVB and BBN are fixed as 0.5 and 0.5 respectively. Results in Table 4 confirm that approximate OrthDNNs via our proposed methods improve generalization by providing effective regularization to large-scale learning.

5.4 Effects of Varying Sample Sizes

We are also interested in the efficacy of SVB and BBN for problems with varying sizes of training samples. To this end, we respectively sample $1/10$, $1/5$, $1/2$, or all of training images per category from ImageNet [46], which constitute our ImageNet training subsets of varying sizes. We train the "ResNeXt-101 (64 $\times$ 4d)" model of [58] in the same way as in Section 5.3 for this investigation. Fig. 4 shows that SVB and BBN consistently improve classification across the regimes from small to large sizes of training samples, and the improvements are more obvious for the smaller ones.

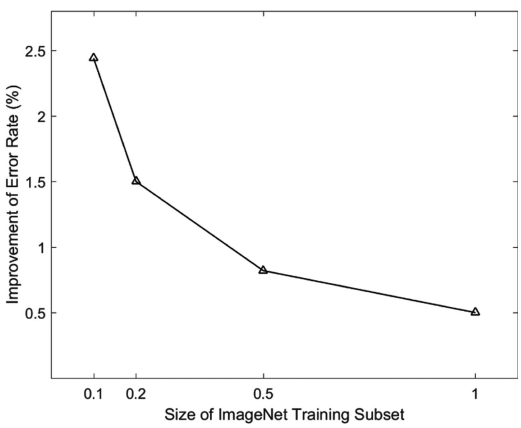


Fig. 4. Regularization effects of our proposed SVB and BBN for varying sizes of training samples. Results in terms of top-1 error rate improvement (%) are obtained by training ResNeXt-101 [58] on ImageNet subsets that are constructed by respectively sampling $1/10$, $1/5$, $1/2$, or all of training images per category from ImageNet. Our methods regularize network training and achieve improved results over the respective baselines of 44.10, 34.21, 25.32, and 19.39 percent. Results are based on single-crop testing of the size 320×320.

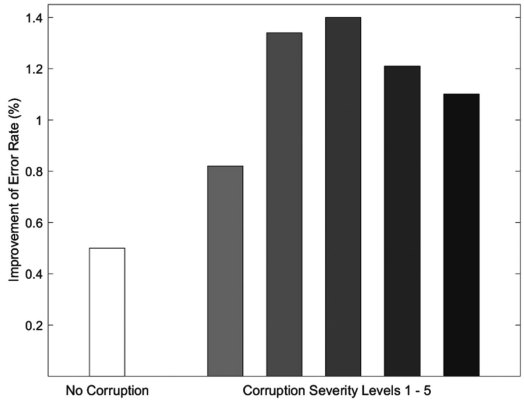


Fig. 5. Robustness test of 5 severity levels on the ImageNet-C dataset [23]. Results in terms of top-1 error rate improvement (%) are obtained by applying ResNeXt-101 [58] models, which are trained without or with regularization of our proposed SVB and BBN, to either clean or corrupted validation images of ImageNet. Our methods give improved results over the respective baselines of 19.39, 30.56, 39.26, 47.09, 58.50, and 70.65 percent.

5.5 Robustness Against Common Corruptions

We have shown in previous experiments that OrthDNNs, particularly our proposed SVB and BBN, have better generalization to testing samples that are drawn from the same distributions of training ones. In this section, we investigate the robustness of OrthDNNs when testing samples are corrupted such that they are getting away from the distributions of training ones. We focus on corruptions that are frequently encountered in natural images, e.g., the "common" corruptions of noise, blur, weather, or digitization [23]. Existing research suggests fragility of deep learning models to corruptions of such kinds [15], and that fine-tuning on specific corruption types would help, but cannot well generalize to other types of corruptions [17], [52]. To this end, we use the ImageNet-C dataset [23] that is produced by applying 15 corruption types of 5 severity levels to validation images of ImageNet [46].[3] As indicated in [23], networks should not be trained or fine-tuned on this dataset for testing of their robustness. We again use the trained models of ResNeXt-101 as described in Section 5.3, with or without the regularization of SVB and BBN. Performance improvements of top-1 error rates are plotted in Fig. 5, where result for each severity level is an average over the 15 types of corruptions. Compared with the result on clean images of ImageNet validation set, Fig. 5 demonstrates better robustness of our proposed methods against common corruptions, and the robustness stands gracefully with the increase of severity levels.

6 Conclusion

In this paper, we present theoretical analysis to connect with the recent interest of spectrally regularized deep learning

3. The 15 types of corruptions include Gaussian Noise, Shot Noise, Impulse Noise, Defocus Blur, Frosted Glass Blur, Motion Blur, Zoom Blur, Snow, Frost, Fog, Brightness, Contrast, Elastic, Pixelate, and JPEG. Refer to [23] for examples of corrupted ImageNet validation images.

methods. Technically, we prove a new generalization error bound for DNNs, which is both scale- and range-sensitive to singular value spectrum of each of networks' weight matrices. The bound is established by first proving that DNNs are of local isometry on data distributions of practical interest, and then introducing the local isometry property of DNNs into a PAC based generalization analysis. We further prove that the optimal bound w.r.t. the degree of isometry is attained when each weight matrix has a spectrum of equal singular values — OrthDNNs with weight matrices of orthonormal rows or columns are thus the most straightforward choice. Based on such analysis, we present algorithms of strict and approximate OrthDNNs, and propose a simple yet effective algorithm called Singular Value Bounding. We also propose Bounded Batch Normalization to make compatible use of batch normalization with OrthDNNs. Experiments on benchmark image classification show the efficacy and robustness of OrthDNNs and our proposed SVB and BNN methods.

Acknowledgments

This work is supported in part by the National Natural Science Foundation of China (Grant No. 61771201), the Program for Guangdong Introducing Innovative and Enterpreneurial Teams (Grant No. 2017ZT07X183), and the Australian Research Council Projects DP180103424, DE190101473, and FL-170100117.

References

[1] P. A. Absil, R. Mahony, and R. Sepulchre, *Optimization Algorithms on Matrix Manifolds*, Princeton, NJ, USA: Princeton Univ. Press, 2007.

[2] M. Arjovsky, A. Shah, and Y. Bengio, "Unitary evolution recurrent neural networks," in *Proc. Int. Conf. Mach. Learn.*, 2016, pp. 1120–1128.

[3] P. Baldi and P. J. Sadowski, "Understanding dropout," in *Proc. Advances Neural Inf. Process. Syst.* 2013, pp. 2814–2822.

[4] N. Bansal, X. Chen, and Z. Wang, "Can we gain more from orthogonality regularizations in training deep networks?" in *Proc. Advances Neural Inf. Process. Syst.*, 2018, pp. 4261–4271.

[5] N. Bansal, X. Chen, and Z. Wang, "Can we gain more from orthogonality regularizations in training deep CNNs?" in *Proc. 32nd Int. Conf. Neural Inf. Process. Syst.*, 2018, pp. 4266–4276.

[6] A. R. Barron, "Universal approximation bounds for superpositions of a sigmoidal function," *IEEE Trans. Inf. Theory*, vol. 39, no. 3, pp. 930–945, May 1993.

[7] P. L. Bartlett, D. J. Foster, and M. J. Telgarsky, "Spectrally-normalized margin bounds for neural networks," in *Proc. Advances Neural Inf. Process. Syst.*, 2017, pp. 6241–6250.

[8] G. Bécigneul, "On the effect of pooling on the geometry of representations," 2017. [Online]. Available: http://arxiv.org/abs/1703.06726

[9] S. Bonnabel, "Stochastic gradient descent on riemannian manifolds," *IEEE Trans. Autom. Control*, vol. 58, no. 9, pp. 2217–2229, Sep. 2013.

[10] O. Bousquet and A. Elisseeff, "Stability and generalization," *J. Mach. Learn. Res.*, vol. 2, pp. 499–526, Mar. 2002.

[11] E. J. Candes and T. Tao, "Decoding by linear programming," *IEEE Trans. Inf. Theor.*, vol. 51, no. 12, pp. 4203–4215, Dec. 2005.

[12] M. Cisse, P. Bojanowski, E. Grave, Y. Dauphin, and N. Usunier, "Parseval networks: Improving robustness to adversarial examples," in *Proc. 34th Int. Conf. Mach. Learn.*, pp. 854–863, 2017.

[13] Y. N. Dauphin, R. Pascanu, Ç. Gülçehre, K. Cho, S. Ganguli, and Y. Bengio, "Identifying and attacking the saddle point problem in high-dimensional non-convex optimization," in *Proc. Advances Neural Inf. Process. Syst.*, 2014, pp. 2933–2941.

[14] L. Dinh, R. Pascanu, S. Bengio, and Y. Bengio, "Sharp minima can generalize for deep nets," in *Proc. Int. Conf. Mach. Learn.*, 2017, pp. 1019–1028.

[15] S. Dodge and L. Karam, "A study and comparison of human and deep learning recognition performance under visual distortions," in *Proc. Int. Conf. Comput. Commun. Networks*, 2017, pp. 1–7.

[16] K. Fukushima, "Neocognitron: A self-organizing neural network for a mechanism of pattern recognition unaffected by shift in position," *Biological Cybern.*, vol. 36, no. 4, pp. 193–202, 1980.

[17] R. Geirhos et al., "Generalisation in humans and deep neural networks," in *Advances Neural Inf. Process. Syst.*, pp. 7548–7560, 2018.

[18] X. Glorot and Y. Bengio, "Understanding the difficulty of training deep feedforward neural networks," in *Proc. Int. Conf. Artif. Intell. Statist.*, 2010, pp. 249–256.

[19] X. Glorot, A. Bordes, and Y. Bengio, "Deep sparse rectifier neural networks," in *Proc. Int. Conf. Artif. Intell. Statist.*, 2011, pp. 315–323.

[20] M. Hardt, B. Recht, and Y. Singer, "Train faster, generalize better: Stability of stochastic gradient descent," in *Proc. 33rd Int. Conf. Mach. Learn.*, 2016, vol. 48, pp. 1225–1234.

[21] K. He, X. Zhang, S. Ren, and J. Sun, "Deep residual learning for image recognition," in *Proc. Conf. Comput. Vis. Pattern Recognit.*, 2016, pp. 770–778.

[22] K. He, X. Zhang, S. Ren, and J. Sun, "Identity mappings in deep residual networks," in *Proc. Eur. Conf. Comput. Vis.*, 2016, pp. 630–645.

[23] D. Hendrycks and T. Dietterich, "Benchmarking neural network robustness to common corruptions and perturbations," in *Proc. Int. Conf. Learn. Representations*, 2019.

[24] G. E. Hinton, N. Srivastava, A. Krizhevsky, I. Sutskever, and R. Salakhutdinov, "Improving neural networks by preventing co-adaptation of feature detectors," *CoRR*, 2012, abs/1207.0580.

[25] K. Hornik, M. Stinchcombe, and H. White, "Multilayer feedforward networks are universal approximators," *Neural Netw.*, vol. 2, no. 5, pp. 359–366, 1989.

[26] G. Huang, Z. Liu, and K. Q. Weinberger, "Densely connected convolutional networks," in *Proc. Conf. Comput. Vis. Pattern Recognit.*, 2017, pp. 4700–4708.

[27] J. Huang, Q. Qiu, G. Sapiro, and R. Calderbank, "Discriminative robust transformation learning," in *Proc. Advances Neural Inf. Process. Syst.* 2015, pp. 1333–1341.

[28] S. Ioffe and C. Szegedy, "Batch normalization: Accelerating deep network training by reducing internal covariate shift," in *Proc. 32nd Int. Conf. Mach. Learn.*, 2015, pp. 448–456.

[29] K. Jia, D. Tao, S. Gao, and X. Xu, "Improving training of deep neural networks via singular value bounding," in *Proc. IEEE Conf. Comput. Vis. Pattern Recognit.*, 2017, pp. 3994–4002.

[30] K. Kawaguchi, "Deep learning without poor local minima," in *Proc. Advances Neural Inf. Process. Syst.*, 2016, pp. 586–594.

[31] K. Kawaguchi, L. P. Kaelbling, and Y. Bengio, "Generalization in deep learning," *CoRR*, 2017, abs/1710.05468.

[32] N. S. Keskar, D. Mudigere, J. Nocedal, M. Smelyanskiy, and P. T. Peter Tang, "On large-batch training for deep learning: Generalization gap and sharp minima," in *Proc. Int. Conf. Learn. Representations*, 2017.

[33] D. Kingma and J. Ba, "Adam: A method for stochastic optimization," in *Proc. Int. Conf. Learn. Representations*, 2015.

[34] A. N. Kolmogorov and V. Tihomirov, "ϵ-entropy and ϵ-capacity of sets in functional spaces," *Amer. Math. Soc. Translations*, vol. 17, pp. 227–364, 2002.

[35] A. Krizhevsky, "Learning multiple layers of features from tiny images," Tech. Rep. Univ. Toroto, 2009.

[36] I. Kuzborskij and C. H. Lampert, "Data-dependent stability of stochastic gradient descent," in *Proc. Int. Conf. Mach. Learn.*, 2018, pp. 2815–2824.

[37] Y. LeCun, L. Bottou, Y. Bengio, and P. Haffner, "Gradient-based learning applied to document recognition," *Proc. IEEE*, vol. 86, no. 11, pp. 2278–2324, Nov. 1998.

[38] Y. LeCun, L. Bottou, G. B. Orr, and K.-R. Müller, "Efficient backprop," in *Proc. Neural Netw.: Tricks Trade*, 1998, pp. 9–50.

[39] C.-Y. Lee, S. Xie, P. W. Gallagher, Z. Zhang, and Z. Tu, "Deeply-supervised nets," in *Proc. 18th Int. Conf. Artif. Intell. Statist.*, 2015, pp. 562–570.

[40] T. Liu, G. Lugosi, G. Neu, and D. Tao, "Algorithmic stability and hypothesis complexity," in *Proc. Int. Conf. Mach. Learn.*, 2017, pp. 2159–2167.

[41] M. Mohri, A. Rostamizadeh, and A. Talwalkar, "*Foundations of Machine Learning*. Cambridge, MA, USA: The MIT Press, 2012.

[42] G. Montúfar, R. Pascanu, K. Cho, and Y. Bengio, "On the number of linear regions of deep neural networks," in *Proc. Advances Neural Inf. Process. Syst.*, 2014, pp. 2924–2932.

[43] P. Orlik and H. Terao, *Arrangements of Hyperplanes*, Grundlehren der mathematischen Wissenschaften. Berlin, Germany: Springer, 1992.

[44] M. Ozay and T. Okatani, "Optimization on submanifolds of convolution kernels in CNNs," *CoRR*, 2016, abs/1610.07008.

[45] J. Pennington, S. S. Schoenholz, and S. Ganguli, "Resurrecting the sigmoid in deep learning through dynamical isometry: Theory and practice," in *Proc. Advances Neural Inf. Process. Syst.*, 2017, pp. 4788–4798.

[46] O. Russakovsky et al., "ImageNet large scale visual recognition challenge," *Int. J. Comput. Vis.*, vol. 115, no. 3, pp. 211–252, 2015.

[47] A. M. Saxe, J. L. McClelland, and S. Ganguli, "Exact solutions to the nonlinear dynamics of learning in deep linear neural networks," in *Proc. Int. Conf. Learn. Representations*, 2014.

[48] H. Sedghi, V. Gupta, and P. M. Long, "The singular values of convolutional layers," in *Proc. Int. Conf. Learn. Representation*, 2019.

[49] K. Simonyan and A. Zisserman, "Very deep convolutional networks for large-scale image recognition," in *Proc. Int. Conf. Learn. Representation*, 2015.

[50] J. Sokolic, R. Giryes, G. Sapiro, and M. R. D. Rodrigues, "Robust large margin deep neural networks," *IEEE Trans. Signal Process.*, vol. 65, no. 16, pp. 4265–4280, Aug. 2017.

[51] I. Sutskever, J. Martens, G. E. Dahl, and G. E. Hinton, "On the importance of initialization and momentum in deep learning," in *Proc. 30th Int. Conf. Mach. Learn.*, May 2013, vol. 28, pp. 1139–1147.

[52] I. Vasiljevic, A. Chakrabarti, and G. Shakhnarovich, "Examining the impact of blur on recognition by convolutional networks," 2016, *arXiv:1611.05760*.

[53] N. Verma, "Distance Preserving Embeddings for General n-Dimensional Manifolds," *J. Mach. Learn. Res.*, vol. 14, pp. 2415–2448, 2013.

[54] S. Wager, S. Wang, and P. S. Liang, "Dropout training as adaptive regularization," in *Proc. Advances Neural Inf. Process. Syst.*, 2013, pp. 351–359.

[55] S. Wang et al., "Analysis of deep neural networks with extended data jacobian matrix," in *Proc. 33nd Int. Conf. Mach. Learn.*, 2016, pp. 718–726.

[56] S. Wisdom, T. Powers, J. R. Hershey, J. L. Roux, and L. Atlas, "Full-capacity unitary recurrent neural networks," in *Proc. Advances Neural Inf. Process. Syst.*, 2016, pp. 4887–4895.

[57] D. Xie, J. Xiongand, and S. Pu, "All you need is beyond a good init: Exploring better solution for training extremely deep convolutional neural networks with orthonormality and modulation," in *Proc. Int. conf. Comput. Vis. Pattern Recognit.*, 2017, pp. 5075–5084.

[58] S. Xie, R. B. Girshick, P. Dollár, Z. Tu, and K. He, "Aggregated residual transformations for deep neural networks," in *Proc. IEEE Conf. Comput. Vis. Pattern Recognit.*, 2017, pp. 5987–5995.

[59] H. Xu and S. Mannor, "Robustness and generalization," *Mach. Learn.*, vol. 86, no. 3, pp. 391–423, 2012.

[60] C. Yun, S. Sra, and A. Jadbabaie, "Global optimality conditions for deep neural networks," in *Proc. Int. Conf. Learn. Representations*, 2018.

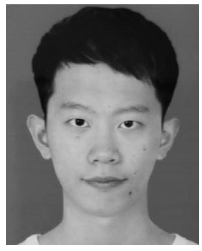
[61] S. Zagoruyko and N. Komodakis, "Wide residual networks," in *Proc. Brit. Mach. Vision Conf.*, 2016, pp. 87.1–87.12.

[62] C. Zhang, S. Bengio, M. Hardt, B. Recht, and O. Vinyals, "Understanding deep learning requires rethinking generalization," in *Proc. Int. Conf. Learn. Representations*, 2017.

[63] C. Zhang, Q. Liao, A. Rakhlin, B. Miranda, N. Golowich, and T. A. Poggio, "Theory of deep learning IIB: Optimization properties of SGD," *CoRR*, 2018, 1801.02254.

Shuai Li received the master of philosophy degree from The Chinese University of Hong Kong, Hong Kong, and the graduate degree from the Special Class for Gifted Young Program in University of Science and Technology of China, Hefei, China. He is currently an associate researcher with the School of Electronic and Information Engineering, South China University of Technology, Guangzhou, China. His research interests include theoretical deep learning, including the formal definition of its architecture, its optimization behaviors, and generalization errors.

Kui Jia received the BEng degree in marine engineering from Northwestern Polytechnical University, Xi'an, China, in 2001, the MEng degree in electrical and computer engineering from the National University of Singapore, Singapore, in 2003, and the PhD degree in computer science from the Queen Mary University of London, London, United Kingdom, in 2007. He is currently a professor with the School of Electronic and Information Engineering, South China University of Technology, Guangzhou, China. His research interests include computer vision, machine learning, and image processing. On these areas, he has authored many publications at prestigious journals and conferences, such as the *IEEE Transactions on Pattern Analysis and Machine Intelligence*, the *International Journal of Computer Vision*, the *IEEE Transactions on Image Processing*, the *IEEE Transactions on Signal Processing*, CVPR, ICCV, and ECCV. His recent research focuses on deep learning and its applications in object recognition, video analysis, and semantic analysis and reconstruction of 3D data.



Yuxin Wen is currently working toward the master/PhD degrees in information and communication engineering, School of Electronic and Information Engineering, South China University of Technology, Guangzhou, China. His research interests include computer vision and machine learning.

Tongliang Liu received the PhD degree from the University of Technology Sydney, Ultimo, Australia. He is currently a lecturer with the School of Information Technologies and the Faculty of Engineering and Information Technologies, and a core member in the UBTECH Sydney AI Centre, at The University of Sydney, Sydney. His research interests include statistical learning theory, computer vision, and optimisation. He has authored and co-authored more than 40 research papers including the *IEEE Transactions on Pattern Analysis and Machine Intelligence*, the *IEEE Transactions on Neural Networks and Learning Systems*, the *IEEE Transactions on Image Processing*, ICML, CVPR, and KDD.

Dacheng Tao (F'15) is professor of computer science and ARC laureate fellow with the School of Computer Science and the Faculty of Engineering and Information Technologies, and the Inaugural director of the UBTECH Sydney Artificial Intelligence Centre, at The University of Sydney, Sydney, Australia. He mainly applies statistics and mathematics to artificial intelligence and data science. His research results have expounded in one monograph and more than 200 publications at prestigious journals and prominent conferences, such as the *IEEE Transactions on Pattern Analysis and Machine Intelligence*, the *IEEE Transactions on Image Processing*, the *IEEE Transactions on Neural Networks and Learning Systems*, T-CYB, IJCV, JMLR, NIPS, ICML, CVPR, ICCV, ECCV, ICDM, and ACM SIGKDD, with several best paper awards, such as the best theory/algorithm paper runner up award in IEEE ICDM07, the best student paper award in IEEE ICDM13, the 2014 ICDM 10-year highest-impact paper award, the 2017 IEEE Signal Processing Society Best Paper Award, and the distinguished paper award in the 2018 IJCAI. He received the 2015 Austrlian Scopus-Eureka Prize and the 2018 IEEE ICDM Research Contributions Award. He is a fellow of the Australian Academy of Science, AAAS, IEEE, IAPR, OSA and SPIE.

▷ **For more information on this or any other computing topic, please visit our Digital Library at** www.computer.org/csdl.